

Sinanabilirlik: Mantıktan Mimariye

ÖFY

August 2026

Özet

Felsefe müfredatının mühendislik pratiğine dokunan kısmı, sanıldığı gibi genel kültür değil, doğrudan araç sağlayan dar bir kümedir. Bu küme dört başlıkta toplanır: biçimsel mantık, eleştirel düşünme, bilim felsefesi ve bilgi felsefesi. Mantık dersleri mühendise yeni bir teknik öğretmez; zaten yaptığı işlemlerin adını ve kurallarını verir, bunun getirisi de adı konmuş hatanın tekrarlanmamasıdır. Buna karşılık mantığın meta-teoremleri gerçek ve aşılamaz bir sınır çizer: statik analiz araçlarının hiçbirisi eksiksiz olamaz, bu bir mühendislik yetersizliği değil ispatlanmış bir teoremdir. Bilim felsefesinin en verimli kısmı Popper’ın yanlışlanabilirlik ölçütü değil, onun yetersizliğini gösteren Duhem–Quine tezi ile Lakatos’un ilerleyen ve çürüten program ayrımıdır; ikincisi teknik borç tartışmasını miktar sorusundan yön sorusuna çevirir. Bilgi felsefesinin merkezî bulgusu, doğru sonuca yanlış gerekçeyle varmanın bilgi sayılmadığıdır; bunun mühendislikteki karşılığı, sebebi açıklanamayan hata düzeltmelerinin sistemde görünmez bir bilgi boşluğu biriktirmesidir. Dil felsefesi ise isimlendirme tutarsızlığı, kavram çakışması ve **null** probleminin ortak kökenini gösterir. Bütün bu başlıkların ortak çıktısı tek bir alışkanlıktır: bir iddianın hangi gözlemle çürütüleceğini sormak.

Giriş

Mühendislik eğitiminde felsefe dersleri genellikle iki uç arasında değerlendirilir. Bir uçta bütün felsefe “genel kültür” sayılır ve pratik karşılığı olmadığı varsayılır. Diğer uçta her felsefi kavramın bir mühendislik karşılığı olduğu iddia edilir ve zorlama benzetmeler kurulur. İki tutum da yanlıştır.

Doğru ayırım şudur: felsefe müfredatının bir kısmı mühendislik pratiğine araç verir, bir kısmı çerçeve verir, kalanı hiçbir şey vermez. Araç veren dersler, ertesi gün uygulanabilecek bir yöntem ya da ayırım sağlar. Çerçeve veren dersler doğrudan uygulanamaz ama kendi mesleğinin varsayımlarını görünür kılar. Kalanı entelektüel olarak değerli olabilir, mesleki gerekçeyle alınması savunulamaz.

Bu ayırımın kurulabilmesi için birkaç terimin baştan tanımlanması gerekir.

Mantık (*logic*), doğru öncüllerden doğru sonuca geçişin kurallarını inceleyen alandır. Yalnızca *tümdengelimli* (*deductive*) çıkarımı güvence altına alır; öncüller doğruysa sonucun kesin doğru olduğu çıkarım türü budur.

Tümevarım (*induction*), tekil gözlemlerden genel bir sonuca geçiştir. Mantıksal olarak geçerli değildir, ancak bilimin ve mühendisliğin fiilen çalıştığı kip budur.

Epistemoloji (*epistemology*), Türkçede bilgi felsefesi ya da bilgi teorisi olarak geçen alandır. İki isim eşanlamlıdır. Konusu, bir inancın ne zaman bilgi sayılacağıdır.

Bilim felsefesi (*philosophy of science*), bilimsel iddiaların hangi koşullarda kabul edileceğini ve bilimsel açıklamanın ne olduğunu inceler.

Yaygın ama yanlış bir kabul, mantık dersinin mühendise “daha iyi kod yazmayı” öğreteceğidir. Öğretmez. Biçimsel mantık, deneyimli bir mühendisin zaten sezgisel olarak uyguladığı ayrımları sembolleştirir. Getirisi teknik değil kavramsaldır: adı olan bir hata, adsız bir hatadan daha az tekrarlanır. Buna karşılık aynı derslerin meta-teorik kısmı gerçekten yeni bilgi verir, çünkü orada mühendisin sezgisiyle ulaşamayacağı bir imkânsızlık ispatı vardır.

Bu not, bir felsefe müfredatının mühendislik açısından değerlendirilmesini kapsar. Kapsamadığı şey, felsefenin kendi içindeki değeridir; bir dersin burada düşük öncelikli sayılması, o dersin değersiz olduğu anlamına gelmez, yalnızca mesleki gerekçeyle savunulamayacağı anlamına gelir.

Biçimsel Mantık

Adlandırma sorunu

Müfredatlarda aynı içerik farklı adlarla açılabilir. “Modern mantık”, “sembolik mantık” ve “matematiksel mantık” terimleri eşanlamlıdır; üçü de Aristotelesçi terim mantığının yerine geçen, sembollerle yazılan sistemi adlandırır. Bir müfredatta hem “Modern Mantık I–II” hem “Sembolik Mantık I–II” bulunuyorsa, bunlar büyük olasılıkla kod değişikliğinden kalan aynı dersin iki kaydıdır; içerik açıklamaları karşılaştırılarak doğrulanmalıdır.

Sembolik mantık, mantığın sembollerle yazılmasından ibarettir. “Kullanıcı giriş yapmışsa ve hesabı aktifse sayfa gösterilir” cümlesi $(G \wedge A) \rightarrow S$ biçiminde yazılır. Sembol kullanmanın gerekçesi matematikteki gerekçeyle aynıdır: uzun cümle karışır, sembol karışmaz. Programlama dillerindeki $\&\&$ ve $!$ operatörleri de aynı işlevi gören sembollerdir.

Önermeler mantığı

Önerme ve bağlaçlar

Önerme (*proposition*), doğru ya da yanlış olabilen cümledir. “Kullanıcı giriş yapmış” bir önermedir; “Dosyayı sil” değildir, çünkü emirler doğruluk değeri almaz. Programlamada mantıksal değer (*boolean*) döndüren her ifade bir önermedir.

Beş bağlaç kullanılır:

Sembol	Okunuşu	Programlama karşılığı
$\neg P$	değil P	!p
$P \wedge Q$	P ve Q	p && q
$P \vee Q$	P veya Q	p q
$P \rightarrow Q$	P ise Q	!p q
$P \leftrightarrow Q$	P ancak ve ancak Q	p == q

Koşul bağlacı $P \rightarrow Q$ sezgiye aykırı davranır. Yalnızca tek bir durumda yanlışdır: P doğru ve Q yanlış olduğunda. Diğer bütün durumlarda doğrudur. Bunun sonucu şudur: P yanlışsa, Q ne olursa olsun $P \rightarrow Q$ doğrudur. “Ay kareyse ben milyonerim” cümlesi doğru bir cümledir, çünkü öncülü yanlıştır.

Bu davranışın programlamadaki doğrudan karşılığı, boş bir koleksiyon üzerinde çalışan döngünün hiç dönmemesi ve dolayısıyla “listedeki her eleman geçerlidir” iddiasının otomatik olarak doğru çıkmasıdır. Boş sepetin ödeme kontrolünden geçmesi bu davranışın ürünüdür.

Doğruluk tabloları

Doğruluk tablosu, bir bileşik önermenin bileşenlerinin bütün değer kombinasyonlarında hangi değeri aldığını gösteren tablodur. n değişken için 2^n satır oluşur.

Pratik değeri buradadır: bir fonksiyon üç mantıksal değer kontrol ediyorsa sekiz durum vardır. Bu sekiz durum yazıldığında, tasarım sırasında düşünülmemiş durumlar görünür hâle gelir.

Somut bir örnek üzerinden gösterilebilir. Bir siparişin onaylanması için üç koşul tanımlanmış olsun: ödeme yapılmış olması, stok bulunması, adres girilmiş olması. İlk yazılan kod tipik olarak yalnızca hepsinin sağlandığı durumu ele alır:

```
if odendi && stokVar && adresVar {  
    onayla()  
}
```

Sekiz satırlık tablo yazıldığında “ödeme alınmış ancak stok yok” satırının özel bir işlem gerektirdiği ortaya çıkar: bu durumda yalnızca onaylamamak yetmez, ödemenin iade edilmesi gerekir. Bu durum tablo yazılmadan fark edilmez.

```
if odendi && !stokVar {  
    paraIadeEt()  
    return  
}  
if odendi && stokVar && adresVar {  
    onayla()  
    return  
}  
hataGoster()
```

Dört mantıksal değer on altı satır üretir. Tablonun okunamayacak kadar büyümesi, kodun fazla karmaşık olduğunun göstergesidir.

Tutarlılık, geçerlilik, eşdeğerlik

Üç kavram ayırt edilir. Bir formül en az bir satırda doğruysa **tutarlıdır** (*satisfiable*); bütün satırlarda doğruysa **geçerlidir** (*valid*); hiçbir satırda doğru değilse **çelişiktir**.

Yazılımdaki karşılıkları nettir. Hiçbir durumda izin vermeyen bir yetkilendirme kuralı çelişiktir, yani bozuktur. Her durumda izin veren bir kural geçerlidir, yani gereksizdir ve kontrol işlevi görmez. İstenen durum tutarlılıktır.

Paket yöneticilerinin bağımlılık çözümü bir tutarlılık kontrolüdür: sürüm kısıtlarının tamamını aynı anda karşılayan bir kombinasyon bulunup bulunmadığı sorulur.

İki formül aynı doğruluk tablosuna sahipse **eşdeğerdir**; biri yerine diğeri yazılabilir. Kod sadeleştirmenin dayanağı budur.

De Morgan kuralları

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q \quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

Sözle ifadesi: değilleme parantezin içine girerken “ve” ile “veya” yer değiştirir, her bileşen ayrıca değillenir.

```
// önce
if !(user != nil && user.Active) { return hata }

// sonra
if user == nil || !user.Active { return hata }
```

İki ifade eşdeğerdir; ikincisi okunabilir. Aynı dönüşüm SQL’de WHERE koşullarını sadeleştirirken ve yetki kontrolü yazarken kullanılır.

Çıkarım kuralları ve geçersiz kalıplar

İki geçerli kural pratikte sürekli kullanılır.

Modus ponens: $P \rightarrow Q$ doğru ve P doğruysa Q doğrudur. “Kod bozuksa test kırılır; kod bozuk; öyleyse test kırılır.”

Modus tollens: $P \rightarrow Q$ doğru ve Q yanlışsa P yanlıştır. “Kod doğruysa test geçer; test geçmedi; öyleyse kod yanlıştır.”

İki geçersiz kalıp bunlara benzediği için sıklıkla karıştırılır.

Sonucu doğrulama (*affirming the consequent*): $P \rightarrow Q$ ve Q doğruysa P doğrudur. Bu geçersizdir. “Kod doğruysa test geçer; test geçti; öyleyse kod doğrudur” çıkarımı yanlıştır, çünkü test o durumu hiç denemiyor olabilir.

Öncülü yalanlama (*denying the antecedent*): $P \rightarrow Q$ ve P yanlışsa Q yanlıştır. Bu da geçersizdir. “Kodu değiştirmedim, öyleyse hata çıkmaz” çıkarımı yanlıştır; bağımlılık güncellenmiş olabilir.

Dijkstra’nın testler hakkındaki formülasyonu doğrudan bu ayrıma dayanır: test hataların varlığını gösterir, yokluğunu asla (Dijkstra, 1970). Kırmızı test bir şey ispatlar, yeşil test ispatlamaz.

Normal formlar

Her mantık ifadesi standart bir kalıba çevrilebilir. **Konjonktif normal form** (CNF), “veya” ile bağlanmış grupların “ve” ile birleştirilmiş hâlidir:

$(a \vee b) \wedge (!a \vee c) \wedge (b \vee !c)$

Disjonktif normal form (DNF) bunun tersidir. Dönüşüm dağıtma kuralıyla yapılır:

$$(a \wedge b) \vee c \equiv (a \vee c) \wedge (b \vee c)$$

CNF’in önemi, karşılanabilirlik çözücülerinin (*SAT solver*) yalnızca bu formatı kabul etmesidir. Her parantez bağımsız bir şart hâline geldiği için hem makine hem insan tarafından ayrı ayrı ele alınabilir.

Bunun günlük karşılığı paket yöneticilerinde görülür. Bir kurulum komutu çalıştırıldığında arka planda sürüm kısıtları mantıksal ifadelere çevrilir ve bir çözücüye verilir. “Çakışan bağımlılık” hatası, çözücünün formülün karşılanamaz olduğunu ispatlaması demektir; rastgele bir başarısızlık değildir.

Yüklemler mantığı

Neden gerekli

Önergeler mantığı cümleleri bölünmez birimler olarak ele alır. Bu yüzden şu çıkarımı temsil edemez:

Bütün insanlar ölümlüdür. Sokrates insandır. Öyleyse Sokrates ölümlüdür.

Üç cümle P , Q , R olarak yazıldığında aralarındaki bağ görünmez olur, çünkü bağ cümlelerin içindedir. Frege'nin 1879'daki *Begriffsschrift* adlı çalışması cümlelerin içine giren bir gösterim kurmuş ve bu tıkanmayı aşmıştır (Frege, 1879).

Yüklem ve niceleyiciler

Yüklem (*predicate*), bir ya da daha çok nesne alıp doğruluk değeri döndüren ifadedir: `insan(sokrates)`, `buyuk(5, 3)`. Programlamada mantıksal değer döndüren fonksiyonlar yüklemidir.

İki **niceleyici** (*quantifier*) kullanılır. $\forall$ “her” anlamına gelir, $\exists$ “en az bir” anlamına gelir.

```
// : hepsi geçerli mi
for _, u := range liste {
    if !u.Gecerli() { return false }
}
return true

// : geçersiz olan var mı
for _, u := range liste {
    if !u.Gecerli() { return true }
}
return false
```

Aralarındaki bağ De Morgan kurallarının genelleştirilmiş hâlidir:

$$\neg \forall x P(x) \equiv \exists x \neg P(x) \quad \neg \exists x P(x) \equiv \forall x \neg P(x)$$

“Hepsi geçerli” ile “geçersiz olan yok” aynı iddiadır. Bu eşdeğerlik, döngü yazarken hangi soruyu sorduğunun bilinçli seçilmesini sağlar.

Boş küme kuralı

Hiç eleman bulunmayan bir küme üzerinde $\forall x P(x)$ *doğru*, $\exists x P(x)$ *yanlıştır*. Gerekçe, “hepsi” iddiasını çürütecek karşı örneğin bulunmamasıdır.

```
liste := []Urun{}
// "hepsi geçerli mi" sorusu true döner
```

Bu davranış mantıksal olarak doğrudur, ancak iş kuralı olarak çoğu zaman istenmez. Boş sepetin geçerli sipariş sayılması, sıfır satırlık dosyanın “bütün satırlar geçerli” testini geçmesi, hiç kalemi olmayan formun onaylanması bu kuralın sonuçlarıdır. Çözüm, boşluk kontrolünün ayrı yapılmasıdır:

```
if len(liste) == 0 {  
    return errBosListe  
}
```

Niceleyici sırası

İki niceleyici yan yana geldiğinde sıra anlamı değiştirir:

$$\forall x \exists y \text{ sever}(x, y) \quad \exists y \forall x \text{ sever}(x, y)$$

Birincisi “herkesin sevdiği biri vardır” der; sevilen kişiler farklı olabilir. İkincisi “öyle bir kişi vardır ki herkes onu sever” der; tek bir kişiden söz edilir. İkinci iddia çok daha güçlüdür.

İç içe döngülerde dış ve iç döngünün hangi niceleyiciye karşılık geldiğini karıştırmak yaygın bir hatadır.

SQL karşılığı

EXISTS doğrudan $\exists$ niceleyicisidir.

```
-- : en az bir siparişi olan müşteriler  
SELECT * FROM customers c  
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);  
  
-- ¬ : hiç siparişi olmayanlar  
SELECT * FROM customers c  
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

NOT IN kullanımında bilinen bir tuzak vardır: alt sorgu tek bir NULL değeri döndürürse sorgu hiçbir satır döndürmez. Sebebi, “ x hiçbirine eşit değil” iddiasının NULL karşısında bilinmez değerini almasıdır. NOT EXISTS bu davranışı göstermez ve tercih edilmelidir.

Klasik mantık

Aristotelesçi mantık yalnızca dört kalıptaki cümlelerle çalışır: “Bütün S’ler P’dir”, “Hiçbir S P değildir”, “Bazı S’ler P’dir”, “Bazı S’ler P değildir”. Temel işlemi **kıyastır** (*syllogism*): iki öncülden bir sonuç türetilir ve türetmenin geçerli olup olmadığı kalıp üzerinden belirlenir.

İkinci konusu tanım teorisidir. Bir kavram, üst kategorisi ve ayırt edici farkı belirtilerek tanımlanır. Bu yapı nesne yönelimli tasarımdaki kalıtım ilişkisinde doğrudan görülür.

Yazılıma dokunan tek gerçek katkısı sınıflandırma kuralıdır: **kategoriler örtüşmemeli ve boşluk bırakmamalıdır**. Bir durum alanı tanımlanırken iki soru sorulur. Birincisi, tanımlanan kategorilere yerleştirilemeyen bir durum düşünülebilir mi? İkincisi, aynı anda iki kategoriye birden düşen bir durum var mı? İki soruya da olumsuz cevap veriliyorsa sınıflandırma sağlamdır.

Bu kontrolün yapılmaması tipik bir bozulma üretir. “Beklemede, aktif, iptal” biçiminde tanımlanmış bir durum alanına, sonradan “iptal edildi ancak ödeme iade edilmedi” durumu geldiğinde, genellikle iptal durumunun yanına ayrı bir mantıksal alan eklenir. Bunun sonucu, şemanın artık anlamsız kombinasyonları da temsil edebilmesidir.

Klasik mantığın sınırı, ilişkileri ve iç içe nicelemeleri ifade edememesidir. “Her sayıdan büyük bir sayı vardır” cümlesi bu dille yazılamaz. Matematiğin biçimselleştirilmesinin önündeki engel budur ve Frege’nin sistemiyle aşılmıştır.

Meta-teoremler ve hesaplanabilirlik sınırı

Bu bölüm, mantık derslerinin mühendise *gerçekten yeni* bilgi verdiği tek yerdir. Diğer konular mevcut sezgiyi adlandırır; burada ise sezgiyle ulaşılamayacak bir imkânsızlık ispatı vardır.

Üç teorem

Gödel’in eksiklik teoremleri (1931): Aritmetiği içeren tutarlı hiçbir biçimsel sistem kendi tutarlılığını kanıtlayamaz ve içinde doğru olduğu hâlde ispatlanamayan önermeler barındırır (Gödel, 1931).

Durma problemi (*halting problem*): Herhangi bir programın duracağını genel olarak belirleyen bir algoritma yoktur. Turing ve Church birbirinden bağımsız olarak 1936’da ispatlamıştır (Turing, 1936).

İspatın yapısı köşegen argümanına dayanır. Böyle bir kontrolcünün var olduğu varsayılır; adı **Durur(p)** olsun ve program duracaksa doğru, sonsuz döngüye girecekse yanlış döndürsün. Şu program yazılır:

```
Kotu():
    eğer Durur(Kotu) doğruysa:
        sonsuza kadar dön
    değilse:
        dur
```

Kontrolcü “duracak” derse program sonsuza döner, “dönecek” derse durur. Her iki durumda da kontrolcü yanılır. Dolayısıyla böyle bir kontrolcü var olamaz.

Rice teoremi: Durma yalnızca bir örnektir. Rice teoremi, bir programın davranışına ilişkin önemsiz olmayan hiçbir özelliğin genel olarak karar verilebilir olmadığını söyler (Rice, 1953). “Bu program hiç çöker mi”, “bu program hiç boş gösterici okur mu”, “bu programın çıktısı hep pozitif midir” sorularının hiçbirini genel olarak cevaplanamaz.

Mühendislikteki sonucu

Statik analiz araçlarının eksiksiz olamamasının sebebi budur. Bu bir araç yetersizliği değil, teorik bir sınırdır.

Sonuç olarak her araç iki seçenekten birini almak zorundadır:

- Bazı gerçek hataları kaçırmak, ancak yanlış alarm vermemek. Yaygın kod denetleyicileri (*linter*) bu yolu izler. Aracın sessiz kalması hata olmadığı anlamına gelmez.
- Hiçbir hatayı kaçırmamak, ancak olmayan hataları da bildirmek. Biçimsel doğrulama araçları bu yolu izler; yanlış alarm oranı yüksek olduğu için benimsenmeleri sınırlı kalır.

Ortası yoktur. Bunun bilinmesi, “daha akıllı bir araç bu hatayı bulur mu” sorusuna harcanan zamanı ortadan kaldırır.

Sınırla yaşama stratejileri

Sınırın kabul edilmesi üç pratik davranış üretir.

Birincisi **katmanlama**. Tek bir araca güvenmek yerine birbirinin kaçırdığını yakalayan katmanlar kurulur: tip sistemi, statik analiz, test, özellik testi ve bulanıklaştırma (*fuzzing*), üretim ortamı izleme. Her katmanın yakalama olasılığı tek başına düşüktür, üst üste konduğunda yükselir.

İkincisi **hataya dayanma**. “Hatasız kod” imkânsız olduğu ispatlandığına göre, hedef hatanın önlenmesi değil hızlı fark edilmesi ve geri alınmasıdır. Özellik anahtarları (*feature flag*), kademeli yayım (*canary deployment*), hızlı geri alma ve iyi tasarlanmış alarm bu hedefe hizmet eder.

Üçüncüsü ve en güçlüsü **problemi daraltma**. Rice teoremi “herhangi bir program için” der. Program uzayı daraltıldığında soru cevaplanabilir hâle gelir.

```
// Genel soru: bu metin geçerli bir e-posta adresi mi?  
func Gonder(email string)
```

```
// Daraltılmış: geçersiz adres bu tipe zaten giremez.  
type Email struct{ deger string }  
func EmailOlustur(s string) (Email, error)  
func Gonder(e Email)
```

İkinci tasarımda doğrulama tek bir noktaya toplanmış ve **Gonder** fonksiyonu için soru ortadan kalkmıştır. Bu, “geçersiz durumu temsil edilemez kıl” ilkesinin biçimsel gereğesidir.

Curry–Howard karşılıklığı

Mantık ile programlama arasında yapısal bir eşleşme vardır. Bu bir benzetme değil, matematiksel bir karşılıklıktır ve bağımsız olarak Curry ile Howard tarafından formüle edilmiştir.

Mantık	Programlama
Önerme	Tip
İspat	O tipte bir değer
$A \rightarrow B$	func (A) B
$A \wedge B$	Çarpım tipi (<i>struct, tuple</i>)
$A \vee B$	Toplam tipi (<i>union, sum type</i>)
Çelişki	Hiçbir değeri olmayan tip

Bunun anlamı şudur: bir fonksiyon yazmak bir teorem ispatlamaktır. **func**(A) B imzasına sahip bir fonksiyon yazılabildiyse, “A varsa B de vardır” önermesi ispatlanmıştır.

Pratik sonucu, tipleri daraltarak ispat yükünün derleyiciye devredilebilmesidir. Yukarıdaki e-posta örneğinde **Gonder** fonksiyonuna geçersiz bir adres geçirilemez; bunun için test yazmaya gerek yoktur, derleyici garanti eder.

İspat yöntemleri

Üç yöntem, algoritma doğrulamasında doğrudan kullanılır.

Doğrudan ispat: Öncülden sonuca adım adım gidilir. “İki çift sayının toplamı çifttir” iddiası şöyle ispatlanır: çift sayılar $2a$ ve $2b$ biçiminde yazılır, toplamaları $2(a + b)$ olur, bu da ikinin katıdır.

Çelişkiyle ispat: İddianın tersi varsayılır ve bir çelişkiye ulaşılır. Kod okurken “burası boş olsaydı yukarıdaki kontrolden geçemezdi” biçiminde yapılan akıl yürütme bu yöntemdir.

Tümevarımla ispat: Sonsuz sayıda durum iki adımda kanıtlanır. Temel durumun doğruluğu gösterilir, ardından herhangi bir durum doğruysa bir sonrakinin de doğru olduğu gösterilir. Domino dizisinin tamamının devrileceğini göstermek için ilk dominonun devrildiğini ve her dominonun bir sonrakini devirdiğini göstermek yeterlidir.

```
func Topla(n int) int {
    if n == 0 { return 0 }
    return n + Topla(n-1)
}
```

Bu fonksiyonun her n için doğru çalıştığı test edilemez, çünkü girdi uzayı sonsuzdur. İspatlanabilir. Temel durum: $n = 0$ için sıfır döner, boş toplam sıfırdır. Tümevarım adımı: **Topla**(k) doğru sonucu veriyorsa **Topla**($k+1$) ifadesi $(k + 1) + (1 + \dots + k)$ değerini hesaplar, bu da istenen sonuçtur.

Döngü değişmezi (*loop invariant*) bu yöntemin günlük hâlidir. Bir döngünün her turunun sonunda doğru kalan koşul belirlenir; döngüye girerken doğru olduğu, her turda korunduğu ve döngü bittiğinde istenen sonucu verdiği gösterilir. Bir döngünün neyi koruduğu belirlenebildiğinde döngü anlaşılmış demektir.

Ozyinelemeli fonksiyonlardaki sonsuz döngü hatalarının büyük kısmı temel durumun yazılmamasından kaynaklanır; tümevarım şemasının bilinmesi bu hatayı önler.

Modal mantık

Klasik mantık iki değerlidir. Modal mantık iki operatör ekler: P (P zorunludur, her durumda doğrudur) ve P (P mümkündür, en az bir durumda doğrudur).

Dağıtık sistemlerde doğrudan kullanılır. Model denetleme (*model checking*) araçları modal ve zamansal mantıkla çalışır ve iki tür özelliği sınar:

- **Güvenlik** (*safety*): kötü bir şeyin hiçbir zaman olmaması. “İki düğüm aynı anda lider olamaz.”
- **Canlılık** (*liveness*): iyi bir şeyin eninde sonunda olması. “Er ya da geç bir lider seçilir.”

Amazon Web Services mühendisleri TLA+ dilini bu amaçla kullandıklarını ve tasarım aşamasında test yoluyla bulunamayacak hatalar tespit ettiklerini belgelemiştir (Newcombe ve diğerleri, 2015).

Çok değerli mantık

Klasik mantık doğru ve yanlış olmak üzere iki değer tanır. SQL üç değer tanır: doğru, yanlış ve bilinmez. Bu, Kleene’nin üç değerli mantığına karşılık gelir.

NULL = NULL	→ bilinmez (doğru değil)
NULL AND false	→ false
NULL AND true	→ bilinmez
NULL OR true	→ true
NOT NULL	→ bilinmez

Bu kurallardan iki sonuç çıkar. Birincisi, WHERE $x = \text{NULL}$ koşulunun hiçbir satır döndürmemesi ve IS NULL kullanılmasının zorunlu olması. İkincisi, yukarıda anlatılan NOT IN tuzağının kaynağının bu üçüncü değer olması.

Aynı yapı başka yerlerde de görülür: Rust dilindeki Option tipi, TypeScript’teki null ile undefined ayrımı, üç durumlu onay kutuları.

Eleştirel Düşünme

Biçimsel mantık yalnızca geçerlilikle ilgilenir. Mühendislik pratiğinin büyük kısmı ise geçerlilikle değil **destek derecesiyle** işler. Eleştirel düşünme, biçimselleştirilemeyen bu alanı kapsar.

Argüman çözümlemesi

Bir iddia üç parçaya ayrılır:

- **İddia:** ne söylendiği.
- **Veri:** neye dayandığı.
- **Varsayım:** veriden iddiaya geçerken örtük olarak kabul edilen şey.

Üçüncüsü belirleyicidir, çünkü hiçbir zaman yazılmaz. Toulmin'in argüman şeması bu yapıyı ayrıntılandırır (Toulmin, 1958).

Bir örnek. Bir değişiklik önerisinde “bu değişiklik uç noktayı yüzde kırk hızlandırıyor” denmiş olsun. Çözümlemesi şöyledir: iddia hızlanma, veri yerel makinede alınan ölçüm, varsayım yerel ölçümün üretim ortamını temsil ettiği. Varsayım incelendiğinde şu görülür: yerel makinede veritabanı aynı makinededir, üretim ortamında ayrı sunucudadır. Darboğaz ağ gecikmesindeyse ve iyileştirme işlemci tarafındaysa üretimde hiçbir değişiklik olmayacaktır.

İddia yanlış değildir, veri de yanlış değildir. Zayıf olan aradaki geçiştir. Denetimin hedefi orasıdır.

Sık karşılaşılan safsatalar

Korelasyonu nedensellikte karıştırmak. “Dağıtımdan sonra hatalar arttı, sebep dağıtımdır.” Aynı gün trafiğin iki katına çıkmış olması aynı gözlemi açıklar.

Post hoc. “Şu satırı değiştirdim, sorun düzeldi.” Yarış durumları (*race condition*) tam olarak bu biçimde “düzeler”; zamanlama değişir, hata saklanır ve sonra geri döner.

Otoriteye başvurma. “Büyük bir teknoloji şirketi böyle yapıyor.” O şirketin çözdüğü ölçek problemi mevcut sistemde bulunmuyorsa, çözüm bir maliyettir.

Hayatta kalan yanlılığı (*survivorship bias*). Başarılı mimari geçişlerin yazısı yazılır, başarısızların yazılmaz. Okunan her başarı anlatısı, görünmeyen başarısızlıkların üzerinde durur.

Kanıtın gücü

Her kanıt eşit değerde değildir. Kanıtın gücü, kaç ihtimali elelediğine bağlıdır. Bu ilkenin en açık uygulaması ikili aramadır.

Yüz deęişiklik arasında birinin hatayı getirdięi biliniyorsa, řüphelenilen deęişikliğe bakmak ortalama elli deneme gerektirir. Ortadaki deęişikliğe bakmak her denemede ihtimallerin yarısını eler ve yedi denemede sonuca ulaşır. Sürüm kontrol sistemlerindeki `bisect` komutunun mantığı budur.

Genel karar kuralı řudur: bir denemeye başlamadan önce, denemenin *iki sonucunun da* bilgi verip vermedięi sorulur.

- Bir kesme noktası konur. Program oraya gelirse bilgi vardır, gelmezse de bilgi vardır. Güçlü hamle.
- Bir kütüphanenin hata kayıtlarına bakılır. Aynı hata bulunursa bilgi vardır; bulunamazsa hiçbir şey öğrenilmez, çünkü kimse bildirmemiş olabilir. Zayıf hamle.

Aynı ilke katman elemesinde de geçerlidir. Uygulama, veritabanı ve ağ arasından hangisinin sorumlu olduęu bilinmiyorsa, uygulamanın tek bir fonksiyonunu incelemek hiçbir katmanı elemez; aynı sorguyu doğrudan veritabanına göndermek tek hamlede katmanların birini eler.

Doęrulama yerine çürütme

En yaygın hata, hipotezi doęrulayacak kanıt aramaktır. Şüpheli görünen ayrıntı her modülde bulunabilir; hiçbir kod tabanı incelendiğinde “burada hiçbir tuhafılık yok” sonucuna varılmaz.

Doęru hamle terstir: hipotezi öldürecek deneme aranır. “Sorun önbellekten kaynaklanıyorsa, önbellek kapatıldığında düzelmelidir.” Bu deneme iki dakika sürer ve hipotezi ya doęrular ya da ortadan kaldırır. Doęrulama arayışı elemez, çürütme denemesi eler.

Bilim Felsefesi

Tümevarım problemi

Yüz gözlemin tamamı bir genellemeyi destekliyorsa, genelleme kanıtlanmış olmaz. “Geçmişte hep böyleydi, gelecekte de böyle olacak” önermesinin gerekçesi sorulduğunda verilecek tek cevap “bu kural geçmişte işe yaradı” olur; bu da tümevarımın tümevarımla gerekçelendirilmesidir, yani döngüdür.

Hume bu problemi 1748’de formüle etmiştir (Hume, 1748) ve genel kabul gören bir çözümü yoktur.

Mühendislikteki karşılığı doğrudandır. Bir sistemin üç aydır sorunsuz çalışıyor olması, yarın çalışacağına garantisi değildir. “Çalışıyor” bir gözlem, “çalışacak” bir tahmindir. İzleme altyapısının varlık sebebi bu ayrımdır.

Yanlışlanabilirlik

Popper’in önerisi, tümevarım probleminin etrafından dolaşmaktır. Teoriler doğrulanamaz ancak yanlışlanabilir. “Bütün kuğular beyazdır” iddiasını hiçbir gözlem kesin olarak kanıtlamaz, ancak tek bir siyah kuğu kesin olarak çürütür. Bu çürütme mantıksal olarak geçerlidir, çünkü modus tollens kuralına dayanır (Popper, 1934).

Buradan sınır ölçütü çıkar: bir iddia bilimselse, onu çürütecek bir gözlem tarif edilebilmelidir.

Mühendislikteki uygulaması, iddiaların denetlenebilirliğidir.

Çürütülemez iddia	Çürütülebilir iddia
“Daha performanslı”	“Aynı yükte %30 daha az işlemci”
“Mimari daha temiz”	“Bu modülde hata sayısı 12’den 3’e indi”
“Bazen yavaşlıyor”	“50 eşzamanlı istekte 5 saniyeyi aşıyor”

Sol sütundaki ifadeler kapatılamaz, çünkü hangi ölçümün onları sonlandıracağı belli değildir. Aynı ölçüt kabul kriterlerine, hizmet seviyesi taahhütlerine ve hata bildirimlerine uygulanır. Yalanlanamayan bir kabul kriteri maddesi bilgi taşımaz.

Duhem–Quine tezi

Yanlışlanabilirlik ölçütü kağıt üzerinde temizdir, uygulamada değildir. Bir teori hiçbir zaman tek başına test edilmez; yanına ölçüm araçlarına, yardımcı varsayımlara ve arka plan bilgisine ilişkin çok sayıda önerme konarak test edilir. Gözlem beklentiyi yalanladığında mantık bir yerde hata olduğunu söyler, nerede olduğunu söylemez (Duhem, 1906; Quine, 1951).

Sonuç şudur: bir teori her zaman kurtarılabilir, yeter ki başka bir varsayım feda edilsin.

Tarihsel örnek ayrımı nettir. Uranüs’ün yörüngesindeki sapma Newton mekaniğini çürütmedi; “görülmemiş bir gezegen daha vardır” varsayımı eklendi ve Neptün bulundu. Merkür’ün yörüngesindeki sapmada aynı hamle yapıldı, varsayılan

gezegene Vulkan adı verildi, ancak böyle bir gezegen bulunamadı; sapmayı açıklayan genel görelilik oldu.

İki durum mantıksal olarak özdeştir. Hangisinin doğru tepki olduğunu mantık belirlemez.

Mühendislikteki karşılığı “yerel ortamda çalışıyor” durumudur. Üretimde ortaya çıkan bir hata kodda, yapılandırmada, veritabanı sürümünde, yük dengeleyicide, ağda ya da bir bağımlılığın alt sürümünde olabilir. Test edilen şey bunların toplamıdır. İkili arama, değişken sabitleme ve asgari yeniden üretim (*minimal reproduction*) çıkarma, bu problemi kaba kuvvetle çözme çabalarıdır.

Bununla bağlantılı tipik hata suçun yanlış yere atılmasıdır. Doğru öncelik sırası kendi kodu, kendi yapılandırması, kendi ortamı ve en son kütüphanedir; çok sayıda kullanıcısı olan bir kütüphanede yeni bir hata bulunmuş olma olasılığı düşüktür.

Teori yüklü gözlem

“Önce gözlem, sonra teori” şeması yanlıştır. Ne aranacağı bilinmeden neye bakılacağı bilinmez. Aynı görüntüye bakan iki uzman farklı şeyler görür, çünkü farklı hipotezlerle bakarlar. Hanson bu olguyu “teori yüklülük” (*theory-ladenness*) terimiyle adlandırmıştır (Hanson, 1958).

Bunun mühendislikteki karşılığı, kayıt dosyası okuma davranışıdır. Veritabanı sorunu aranıyorsa veritabanı satırları okunur ve gözün önündeki zaman aşımı satırı görülmez.

Pratik sonucu şudur: hata aranırken önce hipotez yazılır, sonra o hipotezi *çürütecek* kanıt aranır. Doğrulayacak kanıt aranırsa bulunur, doğru olmasa bile.

Kuhn: paradigma ve devrim

Kuhn, bilimin birikerek ilerlemediğini savunur. Uzun “olağan bilim” dönemlerinde mevcut çerçeve içinde bulmaca çözülür; aykırılıklar biriktiğinde kriz çıkar ve çerçeve bütünüyle değişir (Kuhn, 1962).

Rahatsız edici kısım, yeni çerçevenin eskisinden “daha doğru” değil *farklı* olmasıdır; aralarında ortak bir ölçüt bulunmayabilir. Kuhn bu duruma *eşölçülemezlik* (*incommensurability*) der.

Yazılımdaki karşılığı büyük yeniden yazımlardır. Sakin dönemde mevcut mimari içinde hata düzeltilir ve özellik eklenir. Aykırılıklar birikir: her yeni özellik için geçici çözüm gerekir, basit değişiklikler uzun sürer, sistemin tamamını anlayan kimse kalmaz. Ardından yeniden yazım kararı gelir.

Kuhn’un uyarısı burada da geçerlidir: yeni mimari nesnel olarak daha iyi olmak zorunda değildir. Farklı şeyleri kolaylaştırır, farklı şeyleri zorlaştırır. Mimari geçiş anlatıları çoğu zaman bir problem kümesinin başka bir problem kümesiyle takas edilmesini betimler.

Lakatos: ilerleyen ve çürüyen programlar

Lakatos, Popper ile Kuhn arasında bir konum kurar. Teoriler tek tek değil, *araştırma programları* hâlinde yaşar. Bir programın dokunulmaz bir çekirdeği ve etrafında değiştirilebilir bir koruyucu kuşağı vardır. Gözlem uymadığında çekirdek korunur, kuşak değiştirilir (Lakatos, 1970).

Asıl soru, bu değişikliğin meşru olup olmadığıdır. Lakatos’un ölçütü şudur:

Ekleme yeni bir öngörü üretiyorsa program ilerliyordur; yalnızca mevcut açığı kapatıyorsa program çürüyordur.

Neptün örneğinde ekleme yeni ve sınanabilir bir öngörü üretti: belirli koordinatlarda bir gezegen bulunmalıydı. Gözlem yapıldı ve gezegen bulundu. Vulkan örneğinde aynı biçimdeki ekleme hiçbir yeni öngörü üretmedi ve doğrulanmadı.

Mühendislikteki karşılığı

Bir mimari de bir programdır. Sorulacak soru, yapılan mimari işlerin yeni bir imkân açıp açmadığıdır.

Çürüyen bir programın tipik izi şudur:

Önbellek eklendi, hızlandı.
Önbellek bayat veri veriyor, geçersizleştirme eklendi.
Geçersizleştirme yarış durumu üretiyor, kilit eklendi.
Kilit ölü kilitlenme üretiyor, zaman aşımı eklendi.
Zaman aşımında veri kayboluyor, yeniden deneme eklendi.

Beş adımın hiçbiri yeni bir yetenek getirmemiştir; her biri bir öncekinin ürettiği sorunu kapatmıştır. İlerleyen bir programda ise her adım kendinden sonrasını kolaylaştırır: olay tabanlı yapıya geçildikten sonra denetim kaydının ek maliyetsiz elde edilmesi, yeni bir entegrasyonun kısa sürede tamamlanması, tekrar oynatma yeteneğinin eklenebilmesi.

Ölçüt uygulanabilir bir soruya dönüştürülebilir: son üç ayda yapılan mimari işlerin her biri için “bu, daha önce mümkün olmayan neyi mümkün kıldı” sorusu cevaplanır. Cevapların tamamı “önceki değişikliğin sorununu çözdü” biçimindeyse sorun yamalarda değil çekirdektedir.

Teknik borç kavramının yeniden çerçevesi

Yerleşik görüş teknik borcu tek bir kefiye koyar ve azaltılması gereken bir yük olarak tanımlar. Bu görüş pratikte işlemez, çünkü hiçbir ekip borcu sıfırlayamaz ve sürekli bir erteleme hâli oluşur.

Lakatos’un ayrımı ölçütü değiştirir. Soru “ne kadar borç var” değil “borç hangi yöne gidiyor”dur. Yeni bir imkân açan değişiklik yatırımdır; yalnızca önceki değişikliğin açığını kapatan değişiklik borçtur.

Aynı miktarda düzensiz kod iki farklı durumu betimleyebilir. Birinci durumda son altı ayda yeni bir ödeme sağlayıcısı iki günde eklenmiş, denetim kaydı ek iş olmadan elde edilmiş, yeni bir müşteri tipi tek dosya değişikliğiyle

tanımlanmıştır. İkinci durumda aynı sürede önbellet geçersizleştirmesi düzeltilmiş, yarış durumu yamalanmış, zaman aşımı ayarlanmış ve yeniden denemenin ürettiği yinelenen kayıtlar temizlenmiştir. Borç miktarı aynı, durum farklıdır.

Bu çerçevede iki kural çıkar. Birincisi, ölçütün yalnızca mimari işlere uygulanmasıdır; sıradan özellik ve düzeltme işlerinin yeni kapı açması beklenmez. İkincisi, borcun iki türe ayrılmasıdır. Her hafta gecikmeye yol açan borç ödenmelidir, çünkü ödenmesi hız kazandırır. Aylardır dokunulmayan ve çalışan koddaki düzensizliğin düzeltilmesinin ölçülebilir getirisi yoktur. Uygulanabilir test şudur: bu kalem yüzünden bu ay kaç kez gecikildi.

Bilimsel gerçekçilik tartışması

Teorilerin öne sürdüğü gözlenemez varlıkların gerçekten var olup olmadığı tartışılır.

Gerçekçi konum, teorilerin başarısının en iyi açıklamasının doğru olmaları olduğunu savunur. Karşı konum, tarih boyunca başarılı olmuş pek çok teorelin sonradan yanlış çıktığına dikkat çeker; flojiston ve esir kuramları döneminde işlevseldi. Bu argümana *kötümser tümevarım* denir (Laudan, 1981).

Mühendislikteki karşılığı soyutlamalara duyulan bağlılıktır. Bir etki alanı modelinin “doğru” olduğu savunulurken, aslında yalnızca bugüne kadar işlevsel olduğu bilinmektedir. Son yıllarda “doğru mimari” olarak savunulan yaklaşımların değişmiş olması, mevcut yaklaşımın farklı olacağını düşünmek için gerekçe bırakmaz.

Açıklama nedir

Hempel’in kapsayıcı yasa modeline göre bir olayı açıklamak, onu genel bir yasadan ve başlangıç koşullarından türetmektir (Hempel ve Oppenheim, 1948).

Modelin bilinen sorunu simetridir. Direğin boyundan ve güneşin açısından gölgenin uzunluğu türetilir; bu bir açıklamadır. Gölgenin uzunluğundan direğin boyu da türetilir; bu bir açıklama değildir. Türetim ilişkisi simetrik-tir, açıklama ilişkisi değildir. Demek ki açıklama yalnızca mantıksal türetim değildir; nedensellik gerekir.

Postmortem yazımında aynı sorun görülür. “İşlemci kullanımı yüzde yüze çıktı, bu yüzden istekler zaman aşımına uğradı” cümlesi bir türetim sunar; ancak iki olgu üçüncü bir etkenin sonucu olabilir. Uygulanabilir denetim şudur: bu sebep ortadan kaldırılsaydı olay yine gerçekleşir miydi. Cevap olumluysa bulunan şey kök neden değildir.

Doğa yasası ile olumsal düzenlilik ayrımı

“Bütün bakır örnekleri elektrik iletir” bir yasadır. “Bu odadaki bütün paralar bakırdır” bir düzenliliktir, yasa değildir. İkisi de aynı dilbilgisel yapıdadır. Ayrım, yasanın karşı olgusal durumlarda da geçerli olmasıdır: başka bir bakır örneği getirilse o da iletir.

Mühendislikteki karşılığı ölçümlerin yorumlanmasıdır. “Bu servis her zaman 50 milisaniyenin altında yanıt veriyor” ifadesi bir kısıttan mı kaynaklanmaktadır yoksa mevcut koşulların yan ürünü müdür? Kısıtsa yük arttığında da geçerli

kalır; düzenlilikse trafik değiştiğinde bozulur. Ölçümlerin yük altında tekrarlanmasının gerekçesi budur.

Kuzgun paradoksu

“Bütün kuzgunlar siyahtır” önermesi ile “siyah olmayan her şey kuzgun değildir” önermesi mantıksal olarak eşdeğerdir. Yeşil bir elma ikinci önermeyi doğrulayan bir örnektir. Eşdeğerlik gereği birinci önermeyi de doğrulaması gerekir. Bu sonuç sezgiye aykırıdır ancak mantığı kırılmaz (Hempel, 1945).

Çıkan ders, doğrulama ilişkisinin salt mantıksal biçimle açıklanamayacağı ve kanıtların gücünün eşit olmadığıdır. Bir hipotezin yüz olası konumundan birinin elenmesi zayıf bir kanıttır; yarısının elenmesi güçlü bir kanıttır.

Goodman’ın grue problemi

Goodman şu yüklemi tanımlar: bir nesne, belirli bir tarihten önce gözlemlenirse ve yeşilse ya da o tarihten sonra gözlemlenirse ve maviyse *grue*’dur (Goodman, 1955).

Şimdiye kadar gözlenen bütün zümrütler hem yeşil hem *grue*’dur. Aynı veri iki genellemeyi eşit derecede destekler; genellemeler gelecek hakkında farklı öngörülerde bulunur. Veri seçim yapmaz.

Mühendislikteki karşılığı iki düzeydedir.

Birincisi, hata teşhisidir. “Hata yalnızca pazartesi sabahları çıkıyor” verisi hem “hafta sonu biriken veri işlenirken taşma oluyor” hipotezini hem “pazartesi sabahları yapılan dağıtım bir yarış durumu üretiyor” hipotezini eşit derecede destekler. Hipotez seçimi veriyle değil, önceki deneyimle yapılır. Tehlike, seçimin veriyle yapıldığı sanılmasıdır; bu durumda alternatif hipotez hiç değerlendirilmez.

Doğru yöntem, hipotez kurulduğunda aynı veriye uyan ikinci bir hipotezin aranması ve ardından *iki hipotezi ayıran* gözlemin belirlenmesidir. Yukarıdaki örnekte salı günü elle dağıtım yapmak ya da pazartesi dağıtımını iptal etmek iki hipotezi ayırır.

İkincisi, makine öğrenmesindeki aşırı öğrenmedir (*overfitting*). Eğitim verisine uyan sonsuz sayıda model vardır; hepsi geçmiş veriyi açıklar, gelecekte farklı tahmin üretir. Hangisinin seçileceğini veri belirlemez, model sınıfı ve düzenleme tercihleri belirler.

Mantıksal pozitivizm ve çöküşü

Vienna Çevresi, bir cümlemin ancak deneyle doğrulanabildiği takdirde anlamlı olduğunu savundu. Program üç sebeple çöktü.

Birincisi, ölçütün kendini elemesidir: “yalnızca doğrulanabilir cümleler anlamlıdır” cümlesi deneyle doğrulanamaz. İkincisi, teori yüklü gözlem bulgusudur; saf gözlem yoktur. Üçüncüsü, Duhem–Quine tezidir; tek bir cümle tek başına sınanamaz.

Mühendislikteki yankısı “ölçülemeyen yönetilemez” sloganıdır. Slogan kendi ölçütüne uymaz, çünkü kendisi ölçülebilir bir iddia değildir. Ölçüm değerlidir; ölçülemeyeni yok saymak ise pozitivistin yaptığı hatanın tekrarıdır.

Nedensellik teorileri

“A, B’nin sebebidir” ifadesinin üç farklı çözümlemesi vardır ve üçünün de mühendislikte karşılığı bulunur.

Düzenlilik: A her gerçekleştiğinde B gerçekleşir. Zayıf bir ölçüttür; birlikte görünen ama nedensel olmayan olayları ayırt edemez.

Karşı olgusal bağımlılık: A gerçekleşmeseydi B de gerçekleşmezdi (Lewis, 1973). Postmortem denetiminin dayanağı budur.

Müdahale: A değiştirildiğinde B değişir. En güçlü ölçüttür, çünkü sınanabilir. Özellik anahtarları, A/B testleri ve kademeli yayım bu ölçütün araçlarıdır. Gözlem birlikteliği ipucu verir; müdahale kanıt üretir.

Modeller ve idealleştirme

Bilim, kasten yanlış modeller kullanır: sürtünmesiz yüzey, ideal gaz, noktasal kütle. Soru, yanlış bir modelin nasıl doğru sonuç verdiği.

Cevap, modelin *ne için* yanlış olduğunun bilinmesinde yatar. Sürtünmesiz yüzey modeli, sürtünmenin belirleyici olduğu problemlerde kullanılmaz.

Mühendislikte aynı yapı her yerdedir. Yük testindeki kullanıcı davranışı gerçek davranış değildir, ancak darboğazı gösterir. Sahte nesneler (*mock*) gerçek servis değildir, kasten basitleştirilmiştir. Kapasite planındaki sabit istek süresi varsayımı gerçekte tutmaz, ancak plan işlevini görür.

Uygulanabilir kural şudur: her modelin yanına “bu model hangi koşulda bozulur” notu yazılır. Yazılamıyorsa model anlaşılmamıştır.

Ölçüm teorisi

Bir niceliğe sayı atamak, o sayıya her işlemin uygulanabileceği anlamına gelmez. Stevens’in sınıflandırması dört ölçek düzeyi tanımlar (Stevens, 1946).

Düzye	İzin verilen işlem	Örnek
Sınıflama	Eşitlik	HTTP durum kodu
Sıralama	Büyüklik karşılaştırması	Öncelik: P1, P2, P3
Aralık	Fark	Santigrat sıcaklık
Oran	Oran	Süre, bellek, istek sayısı

Sıralama düzeyindeki verilere ortalama uygulanamaz, çünkü ardışık kategoriler arasındaki uzaklıklar eşit değildir. “Ortalama öncelik 2,4” ifadesi bilgi taşımaz. Aynı hata iş büyüklüğü puanı ortalamalarında, önem derecesi ortalamalarında ve beşli ölçekli memnuniyet anketlerinin ortalamalarında tekrarlanır.

Ortalamanın yerine dağılım raporlanır. “Ortalama 4,2” yerine “katılımcıların yüzde altmış en yüksek değeri, yüzde onu en düşük değeri verdi” ifadesi gerçeği

gösterir. Gecikme ölçümlerinde ortalama yerine yüzdelik dilimlerin kullanılmasının gerekçesi de budur.

Goodhart yasası

Bir ölçüm hedef hâline geldiğinde ölçüm olmaktan çıkar (Goodhart, 1975; Strathern, 1997).

Mekanizma şudur: ölçüm bir durumun belirtisidir. Hedefe dönüştüğünde belirtiyi üretmek, asıl durumu üretmekten kolaydır ve insanlar ölçülen davranışa yönelir. Bu kötü niyet gerektirmez.

Ölçüm	Hedefe dönüştüğünde
Kapsam oranı	Doğrulama içermeyen testler yazılır
Kapatılan kayıt sayısı	Kayıtlar küçük parçalara bölünür
Dağıtım sıklığı	İşlevsiz dağıtımlar yapılır
İlk yanıt süresi	Hızlı ancak çözmeyen yanıtlar verilir
Kapatma süresi	“Yeniden üretilmedi” ile kapatılır

Kapsam oranı örneği açıktır. Aşağıdaki test hiçbir şey doğrulamaz ancak kapsam oranını yükseltir:

```
test('bol calisiyor', () => {  
  bol(10, 2); // doğrulama yok  
});
```

Üç karşı önlem uygulanabilir. Ölçüm izlenir ancak hedefe dönüştürülmez. Her ölçümün yanına onu dengeleyen ikinci bir ölçüm konur: dağıtım hızının yanına üretim hata oranı, kapatılan kayıt sayısının yanına yeniden açılan kayıt oranı, kapsam oranının yanına mutasyon skoru. Ölçüm tek başına değil, açıklamasıyla birlikte raporlanır.

Basitlik ve Occam ilkesi

İki açıklama aynı veriyi açıklıyorsa basit olanın seçilmesi ilkesinin üç farklı gerekçesi vardır.

Pragmatik: Basit açıklamanın sınanması ucuzdur ve yanılma hâlinde kayıp azdır. Bir sistem çöküşünde “bellek doldu” hipotezinin denetlenmesi dakikalar sürer; beş halkalı bir nedensel zincirin denetlenmesi saatler sürer. Basitten başlamak bir arama sırasıdır.

İstatistiksel: Karmaşık model eğitim verisindeki gürültüyü de öğrenir; basit model kalıbı öğrenmek zorunda kalır. Yanlılık–varyans ayrışımı bu davranışı biçimsel olarak açıklar. Burada basitlik bir tercih değil, ölçülmüş bir sonuçtur.

Metafizik: Doğanın gerçekten basit olduğu iddiası savunulamaz.

Kritik nokta şudur: basitlik bir arama stratejisidir, doğruluk kanıtı değildir. Basit açıklama sınamada tutmadıysa “daha basitti” gerekçesiyle ısrar edilmez. Yazılımda “şu an gerekemeyeni ekleme” ilkesinin gerekçesi de basit kodun doğru olması değil, yanlışlığının daha çabuk görünür olmasıdır.

Bilgi Felsefesi

Klasik tanım ve Gettier problemi

Bilginin klasik çözümlemesi üç şart öngörür: önerme doğru olmalı, özne ona inanmalı ve inanç gerekçelendirilmiş olmalıdır.

Gettier, 1963'te üç sayfalık bir yazıyla bu tanımın yetersizliğini gösterdi (Gettier, 1963). Karşı örneklerin ortak yapısı şudur: gerekçe sağlamdır, inanç doğrudur, ancak inancın doğru olmasının sebebi gerekçe değildir; doğruluk tesadüfidir.

Mühendislikteki karşılığı somuttur. Bir servis çalışmadığında kayıt dosyasında bir veritabanı bağlantı hatası görülür ve “veritabanı çökmüş” sonucuna varılır. Veritabanı gerçekten çökmüştür. Ancak görülen kayıt bir önceki güne aittir ve mevcut arızayla ilgisi yoktur. Doğru sonuca yanlış gerekçeyle varılmıştır.

Bu durumun tehlikesi, yöntemin hatalı olduğunun fark edilmemesidir. Sonuç doğru çıktığı için yöntem doğrulanmış sayılır ve bir sonraki teşhiste aynı yöntem uygulanır.

Uygulanabilir ölçüt şudur: bir hata kapatılırken “neden olduğu” açıklanabiliyor mu. Açıklanamıyorsa çözüm tesadüfidir ve sorun geri döner.

Gerekçe zinciri: temelcilik ve bağdaşımcılık

“Neden buna inanıyorsun” sorusu tekrarlandığında zincir ya sonsuza gider, ya döngü oluşturur, ya da bir yerde durur. Bu üçleme Sextus Empiricus'a kadar geri gider ve modern epistemolojide Agrippa üçlemesi olarak anılır.

Temelcilik (*foundationalism*) zincirin durduğunu ve bazı inançların başka gerekçeye ihtiyaç duymadığını savunur. **Bağdaşımcılık** (*coherentism*) zincirin durmadığını, inançların birbirini karşılıklı desteklediğini savunur; yapı piramit değil ağıdır.

Mühendislikte iki yapı da bulunur.

Temelci yapı katmanlıdır. Uygulama kodu kütüphanelere, kütüphaneler standart kütüphaneye, o da işletim sistemine ve donanıma güvenir. En alttaki katman sınanmaz. Zayıflığı, taban bozulduğunda hatanın görünmez olmasıdır; bozuk bellek ya da derleyici hatası durumunda kod doğru, mantık doğru, sonuç yanlıştır ve arama yanlış yerde sürer.

Bağdaşımcı yapı test paketleridir. Hiçbir test tek başına sistemin doğruluğunu göstermez; birlikte bir resim oluştururlar. Zayıflığı, kendi içinde tutarlı ancak bütünüyle yanlış bir ağın mümkün olmasıdır.

```
mockDB.getUser().returns({ id: 1, name: "Ali" });  
expect(getUser(1).name).toBe("Ali");
```

```
mockDB.getUser().returns({ id: 1, name: "Ali" });  
expect(showProfile(1)).toContain("Ali");
```

İki test de geçer ve birbiriyle tutarlıdır. Gerçek veritabanı alan adını farklı döndürüyorsa hiçbirisi gerçekle temas etmemiştir. Bu yüzden test paketinin en

az bir bölümünün gerçek bağımlılıklara dokunması gerekir; kapsayıcı tabanlı entegrasyon testi araçlarının varlık sebebi budur.

İki uygulanabilir soru çıkar. Birincisi: sistemde kanıtsız güvenilen katmanlar hangileridir? Hata bulunamadığında bakılacak yer orasıdır. İkincisi: testler gerçeğe dokunuyor mu, yoksa kendi kendini doğrulayan kapalı bir yapı mı kurulmuş?

İçselcilik, dışsalcılık ve güvenilircilik

İçselcilik (*internalism*), gerekçenin özneye erişilebilir olması gerektiğini savunur: neden bilindiği açıklanabilmelidir. **Dışsalcılık** (*externalism*), açıklanabilirliğin gerekmediğini, sürecin güvenilir olmasının yeterli olduğunu savunur. İkincisinin en yaygın biçimi **güvenilirciliktir** (*reliabilism*): güvenilir bir süreçten gelen inanç bilgidir.

Tartışmanın çözülmemiş olması boş bir çekişme değildir; her konunun haklı olduğu alan farklıdır.

Açıklama gerektiren alan, kararın bir kişiyi olumsuz etkilediği alandır. Bir başvurunun reddedilmesi hâlinde itiraz hakkının kullanılabilmesi için gerekçenin bilinmesi gerekir. Açıklama gerektirmeyen alan, yalnızca sonucun önemli olduğu alandır; bir sıralama algoritmasının neden o sırayı ürettiği sorulmaz.

Ayırt edici soru şudur: yanlışla hâlinde ne olur? Bedel düşük ve geri alınabilirse güvenilirlik yeterlidir; bedel ağır ve geri alınamazsa açıklama zorunludur.

Mühendislikte güvenilirci konum fiilen benimsenir: derleyiciye, tip denetleyicisine ve kütüphanelere nasıl çalıştıkları bilinmeden güvenilir. Bu konumun sınırı, güvenilirliğin nasıl ölçüleceğidir; bir aracın doğruluğu o araç kullanılarak denetlenemez. Thompson'ın derleyiciye yerleştirilen ve kaynak kod incelenerek bulunamayan arka kapı senaryosu bu sınırın uç örneğidir (Thompson, 1984).

Şüphecilik ve sorgulanmayan taban

Şüphecilik, hangi inançların kanıtsız kabul edildiğini görünür kılar. Her bilgi sisteminin sıranmayan bir tabanı vardır: işletim sistemi çağrılarının doğru sonuç verdiği, ağın paketi bozmadığı, diskin yazılımı geri verdiği varsayılır. Bunların sıranması pratik değildir.

Bu güven nadiren kırılır; kırıldığında ise arıza görünmez olur, çünkü aranmayan yeredir. Donanım belleği hataları, disk bit bozulması ve derleyici hataları bu sınıfa girer.

Tanıklık

Bilinen şeylerin büyük kısmı birinci elden değil, başkasından öğrenilmiştir. Tanıklığın (*testimony*) meşru bir bilgi kaynağı olup olmadığı ve hangi koşullarda güvenilir olduğu bu alanın konusudur. Ölçütler kaynağın uzmanlığı, çıkar durumu ve bağımsız doğrulamanın varlığıdır.

Mühendislikteki karşılığı belge ve topluluk kaynaklarına duyulan güvendir. Bir kütüphanenin belgesinde yazan davranış sınanmadan kabul edilir. Güvenilirlik sıralaması genel olarak resmî belge, teknik yazı ve forum cevabı biçimindedir; ancak resmî belge de eskiyebilir ve kaynak kodun okunması her zaman daha güvenilirdir.

Bağlama duyarlılık

Aynı bilgi iddiası bir bağlamda doğru, başka bir bağlamda yanlış olabilir. Bilgi yüklemine uygulanma eşiği bağlama göre değişir; bu konuma *bağlamsalcılık* (*contextualism*) denir.

Mühendislikteki karşılığı “sistem çalışıyor” ifadesidir.

Bağlam	İfadenin anlamı
Günlük toplantı	Bugün alarm gelmedi
Müşteri iletişimi	Hizmet seviyesi taahhüdü karşılanıyor
Denetim	Kanıtlanabiliyor, kaydı mevcut

Üç ifade de doğrudur ve farklı eşiklere karşılık gelir. “Hazır mı” sorusuna cevap verilmeden önce hangi eşikte hazırlık sorulduğunun belirlenmesi, yaygın bir anlaşmazlık kaynağını ortadan kaldırır.

Akran anlaşmazlığı

Aynı kanıta erişimi olan ve benzer yetkinlikteki bir kişi farklı bir sonuca varıyorsa ne yapılmalıdır? Bir konum eşit ağırlık verilmesini, diğeri kendi gerekçesine güvenilmesini savunur.

Mühendislikte uygulanabilir çıkarım şudur: anlaşmazlığın kendisi bir veridir. İki taraf aynı verilere bakıp farklı sonuca varıyorsa veriler yetersizdir; çözüm daha uzun tartışmak değil daha fazla ölçmektir. Farklı verilere bakıyorlarsa önce veriler birleştirilir.

Somut çıkış sorusu şudur: bu tartışmayı hangi ölçüm sonlandırır. Cevap yoksa tartışma teknik değil tercih düzeyindedir ve bir karar verilip uyulması gerekir.

Kolektif bilgi

Bir topluluğun, üyelerinden hiçbirinin tek başına bilmediği bir şeyi bilip bilemeyeceği tartışılır.

Mühendislikteki karşılığı “ekip biliyor” ifadesidir. Bu ifade çoğu zaman kimse bilmediği anlamına gelir. Sınama sorusu şudur: belirli bir kişi ayrılırsa sistemin şu bölümünü kim anlatabilir? Cevap tek bir isimse bilgi ekipte değil o kişidedir.

Bilginin nerede durduğu dayanıklılığını belirler:

Bulunduğu yer	Dayanıklılık
Yalnızca kişide	Kişiyle birlikte kaybolur
Anlık mesajlaşmada	Aranamaz, pratikte kaybolur
Kod yorumunda	Bulunur, ancak eskir
Testte	Bulunur ve eskidiğinde kırılır

Son satır ayrıcalıklıdır: bir davranış belge yerine test olarak yazıldığında hem kayıt altına alınır hem de geçersizleştğinde sinyal üretir.

Epistemik erdemler

Bilgiyi tanımlamak yerine iyi bilen özelliklerini tanımlayan yaklaşım *erdem epistemolojisi* olarak adlandırılır. Mühendislikteki karşılığı şu davranışlardır: kendi hipotezini doğrulamaya değil çürütmeye çalışmak, bilmediğini söyleyebilmek, kanıt karşısında konum değiştirmek, önce kendi kodundan şüphelenmek, sebebi anlaşılmadan düzelen bir sorunu kapatmamak.

Bu liste, işe alım değerlendirmelerinde fiilen ölçülen şeye karşılık gelir: sorulacak asıl soru hangi teknolojinin bilindiği değil, yanılma durumunda ne yapıldığıdır.

Bilginin değeri problemi

Platon *Menon* diyalogunda şu soruyu sorar: doğru inanç zaten işlevini görüyorsa, bilgi neden daha değerlidir? Verdiği örnekte, yolu bilen rehber ile yolu bilmeyen ancak doğru tahmin eden rehber aynı hedefe ulaştırır.

Platon’un cevabı, doğru inancın kalıcı olmadığıdır; gerekçe, inancı yerinde tutar. Cevabın tam tatmin edici olmadığı ve tartışmanın sürdüğü kabul edilir.

Mühendislikteki karşılığı doğrudandır. Tesadüfen düzelen bir arıza ile anlaşılabilir olarak düzelen arıza aynı ara sonucu üretir: sistem çalışır. Fark üç noktadadır. Tesadüfi çözüm tekrarlanamaz, benzer bir arızada yine el yordamı gerekir. Aktarılamaz, çünkü anlatılamaz. Ve sebep ortadan kalkmadığı için geri döner.

Bilginin fazladan değeri dayanıklılık ve aktarılabilirliktir. Postmortem yazımının gerekçesi de budur: sorun çözülmüştür ancak bilgi yazılmazsa kaybolur.

Görünmeyen bilgi boşluğu

Bu ayrımın kurumsal sonucu, dışarıdan gözlenemeyen bir fark üretmesidir. Bir ekip iki farklı hâlde olabilir: sistem çalışmakta ve neden çalıştığı bilinmemekte, ya da sistem çalışmakta ve neden çalıştığı bilinmemekte. Göstergeler her iki durumda aynıdır.

Fark ancak arıza anında ortaya çıkar. Bu nedenle boşluk, kriz anına kadar ölçülmez.

Boşluğun birikmesinin sebebi, tekil tesadüfi çözümlerin her birinin makul görünmesidir. Biriktğinde sistemin bir bölümü kimsenin anlamadığı kurallarla ayakta kalır ve o bölüme dokunulmaz hâlde gelir. Bu bölge çoğunlukla “hassas” olarak adlandırılır; doğru adlandırma “anlaşılmamış”tır.

Yapısal sebep ödöl düzeninde yatar. Bir kaydın kapatılıp kapatılmadığı ölçölür, neden kapandığı ölçölmez. Anlamak zaman alır, anlamamak almaz ve ikisi aynı kaydı kapatır.

Üç gösterge boşluğun büyüklüğü hakkında fikir verir: yeniden açılan kayıt oranı, sebebi açıklanamayan çözümlerin sıklığı ve kod tabanında kimsenin dokunmadığı bölgelerin haritası.

Uygulanabilir tek kural, kayıt kapatılırken sebebin yazılmasının zorunlu kılınmasıdır. Yazılamıyorsa kayıt geçici olarak işaretlenir. Maliyeti bir cümledir; karşılığında tesadüfi çözümler sayılabilir hâle gelir.

Aynı sorun kod yazımında da geçerlidir. Deneme yanılmayla yazılmış ve neden çalıştığı yazarınca bilinmeyen kod, doğru yazılmış koddan ayırt edilemez; ikisi de aynı yeşil test sonucunu üretir.

Nasıl bilgisi ve neyi bilgisi

Ryle iki bilgi türünü ayırır: *neyi* bilgisi (*knowing-that*) önermesel bilgidir, söylenebilir ve aktarılabilir; *nasıl* bilgisi (*knowing-how*) beceridir, tam olarak dile dökülemez (Ryle, 1949). Polanyi bu ikinciye *örtük bilgi* (*tacit knowledge*) olarak adlandırır ve bildiğimizin söyleyebildiğimizden fazla olduğunu savunur (Polanyi, 1966).

Ayrım, belgelendirmenin neden yetersiz kaldığını açıklar.

Neyi bilgisi (yazılabilir)	Nasıl bilgisi (yazılamaz)
Arayüzün parametreleri	Hangi durumda hangisinin kullanılacağı
Mimari şeması	Nereye dokunulmaması gerektiği
Kodlama standartları	Bir kodun sorunlu olduğunu sezilmesi
Dağıtım adımları	Ne zaman dağıtım yapılmayacağı

Sonuçları şunlardır. Yeni katılan kişi belgeleri okusa da bir süre yavaş çalışır; eksik olan nasıl bilgisidir. Birlikte programlama belge yazmaktan daha etkilidir, çünkü nasıl bilgisi ancak birlikte çalışılarak aktarılır. Kod incelemesinde “bu bana yanlış geliyor ancak gerekçesini yazamıyorum” ifadesi meşrudur.

Ayrımın olumsuz yüzü de vardır: nasıl bilgisi sorgulanamadığı için yanlış olduğunda düzeltilmesi zordur. “Biz hep böyle yaparız” ifadesinin arkasında çoğunlukla bu tür bir bilgi bulunur.

Dil Felsefesi

Anlam ve gönderge ayrımı

Frege şu bulmacadan yola çıkar: “sabah yıldızı akşam yıldızıdır” cümlesi bilgi verir, “Venüs Venüs’tür” cümlesi vermez. İkisi de aynı gök cismine gönderme yapmaktadır.

Çözüm iki katmanın ayrılmasıdır. **Gönderge** (*Bedeutung, reference*) ifadenin işaret ettiği nesnedir. **Anlam** (*Sinn, sense*) o nesnenin veriliş biçimidir (Frege, 1892). İki ifade aynı göndergeye sahip olup farklı anlamlar taşıyabilir; bilgi bu farktan doğar.

Adlandırmanın tarafsız olmaması

Ayrımın en görünür sonucu, aynı göndergeye farklı adlar verildiğinde anlaşmazlığın gönderge düzeyinde değil anlam düzeyinde ortaya çıkmasıdır. Taraflar aynı şeyden söz ettiklerini kabul eder ve yine de hiçbir noktada uzlaşmazlar.

Bu durum haber kuruluşlarının yazılı dil politikalarında somutlaşmıştır. BBC’nin yayın ilkelerinde “terörist” teriminin anlamayı sağlayan değil engelleyen bir ifade olabileceği belirtilir ve faili tanımlayan terimlerin tercih edilmesi istenir: bombacı, saldırgan, silahlı kişi, isyancı, militan. Gerekçe, izleyiciye olguların sunulup değerlendirilmesinin ona bırakılmasıdır. Kurumun dünya işleri editörü terimi yüklü bir kelime olarak tanımlar ve ahlaken onaylanmayan bir yapı için kullanıldığını belirtir.

Associated Press’in üslup kılavuzu benzer bir kural içerir: terim doğrudan alıntı ya da yetkililere atıf dışında belirli eylem ve gruplar için kullanılmaz, bunun yerine eylemin kendisi tanımlanır. Aynı kılavuz, hangi devletlerin hangi örgütü nasıl sınıflandırdığını olgu olarak aktarır; bu, kurumun kendi sesi ile aktarılan olgunun ayrılmasıdır. Kılavuzda yer adlarına ilişkin ayrı bir kural da bulunur.

Politikalar her iki yönden de eleştirilir. Bir yandan aynı kuruluşların başka olaylarda aynı terimi kullandığı gösterilerek tutarsızlık iddia edilir; öte yandan başka coğrafyalarda aynı politikanın uygulanması yerel tepki üretir. Keşmir’e ilişkin haberlerde farklı ajansların farklı yer adlandırmaları kullanması aynı olgunun örneğidir.

Üslup kılavuzlarının etkisi kurumsal sınırları aşar. AP kılavuzunun binlerce yayın kuruluşuna dağıtılması nedeniyle bir terim değişikliği zincirleme etki üretir; göçmenlik terminolojisindeki bir değişikliği büyük gazetelerin izlemesi bu etkinin belgelenmiş örneğidir. Politikaların çoğu belirli olayların ardından yazılmıştır.

Buradan çıkan sonuç, Frege’nin ayrımının doğal uzantısıdır: tarafsız ad yoktur. Tarafsız sayılan ad da bir seçimin ürünüdür.

Yazılımdaki karşılığı

Aynı veri farklı katmanlarda farklı adlarla bulunabilir: veritabanında bir alan adı, arayüzde başka bir ad, kayıt dosyasında üçüncü bir kısaltma. Gönderge

aynıdır, adlar farklıdır ve yeni katılan kişi bu özdeşliği bilmez. Sonuç, birleştirme hatalarında ve arama yaparken kaybedilen zamanda görülür. Çözüm, kavramların merkezî bir sözlükte tanımlanmasıdır.

İkinci karşılık kimlik ve değer karşılaştırmasıdır: eşitlik operatörünün göndergeyi mi yoksa içeriği mi karşılaştırdığı her dilde ayrıca kararlaştırılan bir konudur.

Anlamanın kullanımda oluşması

Wittgenstein erken döneminde dilin dünyanın resmi olduğunu savunmuş, geç döneminde bu görüşü terk etmiştir. Yeni konum, bir kelimenin anlamının dildeki kullanımı olduğudur (Wittgenstein, 1953).

Dayanağı “oyun” örneğidir. Futbol, satranç, tek kişilik kart oyunu ve çocukların koşuşturması oyun sayılır. Bütün örneklerde ortak tek bir özellik bulunmaz: bazılarında rekabet, bazılarında kazanan, bazılarında kural yoktur. Wittgenstein bu duruma *aile benzerliği* (*family resemblance*) der; örnekler birbirine benzer, ortak bir öz taşımazlar.

Yazılımdaki karşılığı kavram çakışmasıdır. “Aktif kullanıcı” ifadesi pazarlama biriminde son otuz günde giriş yapmış kişiyi, faturalama biriminde ödemesi güncel olan kişiyi, altyapı tarafında oturumu açık olan kişiyi ifade edebilir. Üç birim aynı terimi kullanır ve farklı sayılar raporlar.

Tek bir doğru tanım arayışı sonuçsuz kalır, çünkü böyle bir tanım yoktur. Etki alanı odaklı tasarımdaki *sınırlı bağlam* (*bounded context*) kavramı bu bulgunun mühendislik karşılığıdır (Evans, 2003): her bağlamda ayrı ad ve ayrı tanım kullanılır.

Söz edimleri

Austin, bazı cümlelerin bir durumu betimlemediğini, bir eylemi gerçekleştirdiğini gösterir (Austin, 1962). “Yağmur yağıyor” betimler ve doğruluk değeri alır. “Söz veriyorum” betimlemez; söylendiği anda söz verme eylemi gerçekleşir.

Bu cümleler doğru ya da yanlış değil, *başarılı* ya da *başarısızdır*. Başarı koşulları vardır: yetkisiz bir kişinin resmî bir ilan cümlesi kurması hiçbir sonuç doğurmaz.

Yazılımdaki karşılığı komut ile sorgu ayrımıdır. Bilgi döndüren çağrılar ile durum değiştiren çağrılar farklı türden ifadelerdir; komut-sorgu ayrımı ve olay kaynaklı mimarilerdeki olay adlandırması bu ayrımı kodlar.

Pratik sonucu adlandırmadadır. Sorgu izlenimi veren bir adın altında durum değiştiren bir işlem bulunması, yeniden deneme mekanizmalarında ve hata ayıklama sırasında beklenmedik sonuçlar üretir. Bilgi döndürenler için *get*, *find*, *is* gibi örnekler; durum değiştirenler için *create*, *update*, *delete* gibi örnekler kullanılır ve durum değiştirenler mümkün olduğunca tekrarlanabilir (*idempotent*) tasarlanır.

Örtük anlam

Grice, söylenene iletilenin farklı olduğunu ve aradaki farkın kurallı biçimde üretildiğini gösterir (Grice, 1975). “Bazı testler geçti” cümlesi sözlük anlamıyla en az bir testin geçtiğini söyler ve hepsinin geçmiş olmasıyla uyumludur. Buna rağmen dinleyici bazılarının geçmediğini anlar, çünkü hepsi geçseydi konuşmacının bunu söylemesi beklenirdi.

Grice bu çıkarımı işbirliği ilkesi ve ona bağlı kurallarla açıklar: gerektiği kadar bilgi verilir, doğru olan söylenir, konuyla ilgili kalınır, açık olunur.

Yazılımdaki karşılığı iki yerededir. Birincisi hata mesajlarıdır: bilgi taşımayan bir hata metni, bu kuralların ihlalidir ve kullanıcıyı belgeye yönlendirir. Çözümü içeren bir hata mesajı belge okuma ihtiyacını ortadan kaldırır.

Zayıf: `Error: invalid configuration`

Güçlü: `Error: 'timeout' saniye cinsinden bir sayı olmalı`
(örn. 30). Alınan değer: "30s"

İkincisi imzaların örtük vaatleridir. Bir fonksiyon imzası yalnızca ne döndürdüğünü söyler; sıralı olup olmadığı, tamamını mı yoksa sayfalanmış bir bölümünü mü döndürdüğü, silinmiş kayıtları içerip içermediği yazılı değildir. Kullanıcılar bu varsayımlarla kod yazar. Varsayım değiştiğinde imza aynı kaldığı için derleyici uyarı vermez. Çözüm, varsayımın ada ya da teste taşınmasıdır.

Belirli betimlemeler

Russell şu cümleyi ele alır: “Fransa’nın şimdiki kralı keldir.” Fransa’da kral bulunmadığı için cümle doğru değildir. Yanlış kabul edildiğinde ise değillesinin doğru olması gerekir; “Fransa’nın şimdiki kralı kel değildir” cümlesi de doğru değildir. Bir cümle ve değillesinin ikisinin birden yanlış olması iki değerli mantıkta mümkün değildir.

Russell’in çözümü cümlelerin mantıksal yapısını açmaktır (Russell, 1905). Cümle üç iddianın birleşimidir: bir Fransa kralı vardır, tektir ve keldir. Birinci bileşen yanlış olduğu için bileşke yanlıştır. Değilleme ise “bu üç bileşenden en az biri yanlıştır” anlamına gelir ve doğrudur; çelişki ortadan kalkar.

Genel bulgu şudur: bir cümlelerin yüzey yapısı mantıksal yapısı olmayabilir. “Fransa’nın kralı keldir” cümlesi “Ali keldir” cümlesiyle aynı biçimdedir; ancak birincisi örtük bir varlık iddiası taşır.

Yazılımdaki karşılığı boş gösterici problemidir.

```
user := GetUser(999)      // böyle bir kayıt yok
fmt.Println(user.Name)    // örtük varlık iddiası
```

İkinci satır tek bir işlem gibi görünür, ancak “böyle bir kayıt vardır” iddiasını içerir. Hoare boş gösterici kavramını sonradan “milyar dolarlık hata” olarak nitelmiştir (Hoare, 2009).

Rust’taki `Option`, Haskell’deki `Maybe` ve Go’daki iki değerli dönüş kalıbı Russell’in çözümünü aynı yapıdadır: varlık iddiası ayrı bir katmana çıkarılır ve denetlenmeden ilerlenemez.

Belirsizlik ve muğlaklık

İki farklı sorun karıştırılır. **Belirsizlik** (*ambiguity*) bir ifadenin birden çok anlam taşımasıdır. **Muğlaklık** (*vagueness*) tek anlamlı bir ifadenin sınırının net olmamasıdır; “uzun boylu” ifadesinin nerede başladığı belirsizdir.

Muğlaklığın uç biçimi yığın paradoksudur: bir kum yığınının tek tane çıkarmak yığını ortadan kaldırmaz, ancak işlem yinelendiğinde yığın kalmaz. Her adım savunulabilir, sonuç savunulamaz.

Yazılımdaki karşılıkları ayırır. Belirsizlik, aynı adın farklı bağlamlarda farklı anlaşılmasıdır. Muğlaklık, gereksinim metnindeki ölçüsüz ifadelerdir: “yavaş”, “büyük dosya”, “yüksek trafik”. İki kişi aynı kaydı okuyup farklı sonuç çıkarır ve düzeltmenin yeterli olup olmadığı tartışılır.

Muğlaklık her zaman kusur değildir; erken karar vermemek için bilinçli olarak korunabilir. Belirleyici olan, muğlaklığın kasıtlı mı yoksa gözden kaçmış mı olduğudur.

Özet eşleme

Dil problemi	Mühendislikteki karşılığı
Anlam–gönderge ayrımı	İsimlendirme tutarsızlığı, kimlik/değer karşılaştırması
Sabit tanımın olmaması	Kavram çakışması, sınırlı bağlam
Söz edimleri	Komut–sorgu ayrımı, adlandırma
Örtük anlam	Yazılmamış varsayımlar, hata mesajları
Belirli betimlemeler	Boş gösterici problemi, <code>Option</code> tipi
Muğlaklık	Ölçüsüz gereksinim metinleri

Anlatı ve Öznellik

Bir olayın kendisi ile o olayın anlatısı ayrı şeylerdir. Anlatan kişi nereden başlayacağını, neyi atlayacağını ve hangi sırayı izleyeceğini seçer. Aynı olaylardan farklı anlatılar çıkabilir ve hiçbirisi yalan içermeyebilir.

Mühendislikteki karşılığı postmortem metnidir. Kayıt dosyaları olguları içerir; postmortem bir anlatıdır ve aradaki fark seçimdir.

Başlangıç noktası sebebi belirler

Bir anlatının nereden başlatıldığı, hangi etkenin sebep sayılacağını belirler. Bir sistem çöküşü için:

- Anlatı olay saatinde başlarsa sebep hatalı dağıtımdır.
- Anlatı üç ay öncesinden başlarsa sebep test ortamının üretim ortamına benzememesidir.
- Anlatı bir yıl öncesinden başlarsa sebep ekibin yeterince büyütülmemesidir.

Üçü de savunulabilir. Dolayısıyla kök neden bir keşif değil, bir seçimdir. Bu, Hempel'in açıklama probleminin anlatı düzeyindeki karşılığıdır.

Anlatıcının konumu ve geriye dönük yanlılık

Postmortem'i yazan kişi çoğunlukla olayın içindedir ve kendi kararlarını anlatır. Kasıt gerekmez; kendi bakış açısından bazı etkenler görünmez.

İkinci sorun geriye dönük yanlılıktır. Anlatı, sonuç bilindikten sonra kurulur; bu yüzden ara adımlar kaçınılmaz görünür ve “şu uyarı görülmeliydi” değerlendirmesi yapılır. Olay anında çok sayıda uyarı arasından hangisinin belirleyici olduğu bilinmiyordu. Suçlamasız postmortem yaklaşımının gerekçesi, o an bilinenle sonradan bilinenin ayrılmasıdır (Dekker, 2002).

Öznellik tartışmasının ölçülü hâli

Sert konum yalnızca anlatıların bulunduğunu, doğru bir anlatı olmadığını savunur. Ölçülü konum, olguların gerçek olduğunu ancak hangi olguların seçilip nasıl bağlanacağını yoruma açık olduğunu savunur. İkincisi savunulabilir: bir kayıt satırının varlığı tartışmalı değildir, “asıl sebep buydu” cümlesi ise bir yorumdur.

Test Yöntemleri ve Araç Ekosistemi

Yeşil testin hiçbir şey ispatlamaması bulgusu, testlerin kırılabilir hâle getirilmesi sorusunu doğurur. Bu bölüm, o sorunun yönetsel ve araçsal cevabını içerir.

Testi kırılabilir kılan yöntemler

Sınır değerleri

Hatalar değer aralığının ortasında değil kenarlarında bulunur. Bir ile yüz arasında değer alan bir fonksiyon için sınanacak değerler sıfır, bir, iki, doksan dokuz, yüz ve yüz birdir; elli hiçbir bilgi vermez. Koleksiyonlar için boş koleksiyon, tek elemanlı koleksiyon ve çok elemanlı koleksiyon ayrı ayrı sınanır.

Hata yolları

Yaygın uygulama başarılı durumun sınanmasıdır; o durum zaten çalışmaktadır. Sınanması gereken durumlar kaydı bulunamaması, boş değer gelmesi, boş metin gelmesi ve bağımlılığın erişilemez olmasıdır.

Mutasyon testi

En doğrudan denetim yöntemi kodun kasıtlı olarak bozulmasıdır. Bir karşılaştırma operatörü değiştirilir, bir mantıksal bağlaç ters çevrilir, bir satır silinir ve testler çalıştırılır. Testler hâlâ geçiyorsa o testler işlev görmüyordur.

Bu işlemi otomatikleştiren araç sınıfına *mutasyon testi* denir. Elle uygulanması da mümkündür ve birkaç dakika sürer.

Kapsam oranının sınırı

Kapsam oranı, satırların ne kadarının çalıştırıldığını gösterir; ne kadarının doğru olduğunu göstermez. Doğrulama içermeyen bir test kapsam oranını yükseltir ve hiçbir şey sinamaz. Kapsam, nereye bakılmadığını söyleyen bir göstergedir.

Özellik testi

Tek tek örnekler yerine bir değişmez kural yazılır ve araç çok sayıda girdi üretir. “Bir koleksiyon sıralandığında eleman sayısı değişmez” biçiminde bir kural, yazarın aklına gelmeyecek girdilerle sınanır.

Modern özellik testi kütüphanelerinin ortak özelliği, kırıcı bir girdi bulduklarında onu en küçük hâline indirgemeleridir (*shrinking*). Bu davranış tek bir kütüphaneye özgü değildir; alanın standart özelliğidir.

Gerçekçi veri

Test verileri genellikle temizdir, gerçek veri değildir. Sınanması gereken girdiler arasında dile özgü karakterler, birleşik karakterler, çok uzun metinler, baştaki ve sondaki boşluklar, tırnak ve yorum işaretleri ile işaretleme etiketleri bulunur.

Türkçe karakterlerin büyük–küçük harf dönüşümündeki davranışı buna tipik bir örnektir.

Önce kırılan test

Bir hata bulunduğunda sıra şudur: hatayı gösteren test yazılır, testin *kırıldığı* görülür, kod düzeltilir, testin geçtiği görülür. İkinci adım atlanamaz; test kırılmadıysa yanlış şey sınıyordu. Bu yöntemle test paketi zamanla gerçek hatalardan oluşan bir koleksiyona dönüşür.

Sıra ve zamanlama

Testlerin karışık sırada çalıştırılması, testler arası gizli bağımlılıkları ortaya çıkarır. Eşzamanlılık içeren dillerde yarış durumu tespit edicilerinin kullanılması ayrıca gereklidir.

Kategorilere göre araç ekosistemi

Aşağıdaki tablolar kategori bazlı bir haritadır. İçerdikleri bilgi zaman içinde eskir; kritik kararlarda güncel durum doğrulanmalıdır.

Test çatısı ve kapsam

Dil	Test çatısı	Kapsam
Go	testing (dahili)	go test -cover (dahili)
JS / TS	Vitest, Jest, node:test	c8, Istanbul
Python	pytest, unittest	coverage.py
Java	JUnit 5, TestNG	JaCoCo
C#	xUnit, NUnit	Coverlet
Rust	cargo test (dahili)	cargo-llvm-cov
PHP	PHPUnit, Pest	PHPUnit + Xdebug
Ruby	RSpec, Minitest	SimpleCov
C / C++	GoogleTest, Catch2	gcov + lcov
Kotlin	Kotest, JUnit 5	Kover
Swift	XCTest	Xcode dahili
Elixir	ExUnit (dahili)	ExCoveralls
Scala	ScalaTest, MUnit	scoverage
Haskell	HSpec, tasty	HPC
Dart	package:test (dahili)	dart test --coverage
Erlang	EUnit, Common Test	cover (dahili)
Clojure	clojure.test	Cloverage
OCaml	Alcotest	bisect_ppx
R	testthat	covr
Bash	bats-core	kcov
SQL	pgTAP, tSQLt	—
Solidity	Foundry, Hardhat	solidity-coverage

Mutasyon testi

Dil	Araç
Java	PIT
JS / TS	Stryker
C#	Stryker.NET
Scala	Stryker4s
Python	mutmut, cosmic-ray
PHP	Infection
Ruby	Mutant
Rust	cargo-mutants
C / C++	Mull
Go	gremlins, go-mutesting türevleri, gomutant
Swift	muter

Java ekosisteminde PIT, web ekosisteminde Stryker olgunluk açısından öne çıkar; ikisi de aktif olarak geliştirilir ve özellik açısından zengindir.

Go tarafındaki durum daha karışıktır ve sıklıkla olduğundan zayıf tarifiedilir. Gremlins etkin biçimde sürdürülmekte ve PIT'ten esinlenerek tasarlanmış olmakla birlikte kararlı sürüm öncesinde bulunmakta, geriye dönük uyumluluk taahhüdü vermemekte ve büyük modüllerde çalışma süresi ciddi biçimde uzamaktadır. Eski araçlardan türeyen çatallar sürekli tümleştirme için eşik denetimi ve değişiklik bazlı çalışma gibi özellikler eklemiştir; önbellekleme ve yalnızca değişen satırları sınama gibi hız özellikleriyle yeni araçlar ortaya çıkmıştır. Sonuç olarak ekosistem Java ve JavaScript kadar olgun değildir, ancak yok sayılamaz.

Özellik testi

Dil	Araç
Haskell	QuickCheck, Hedgehog
Python	Hypothesis
JS / TS	fast-check
Scala	ScalaCheck
Java	jqwik
C#	FsCheck, CsCheck
Rust	proptest, quickcheck
Go	rapid, gopter (testing/quick yetersiz)
Kotlin	Kotest property
Elixir	StreamData
Erlang	PropEr
Clojure	test.check
OCaml	QCheck
PHP	Eris
Ruby	PropCheck, Rantly
C++	RapidCheck
Swift	SwiftCheck

Yaklaşımın kökeni Haskell için geliştirilen QuickCheck kütüphanesidir (Claessen ve Hughes, 2000); diğer dillerdeki kütüphanelerin adlarındaki ortaklık bu kökenden gelir.

Go'nun standart kütüphanesindeki `testing/quick` paketi yetersizdir: veri üretimi için yeterli olanak sunmaz ve kırıcı girdiyi küçültme özelliği bulunmaz. Buna karşılık Go 1.18 sürümünden itibaren dilin kendi bulanıklaştırma desteği (`go test -fuzz`) mevcuttur ve bu ayrı bir kategoridir.

Diğer kategoriler

Kategori	Başlıca araçlar
Sahte nesne	Mockito (Java), Moq (C#), <code>unittest.mock</code> (Python), Sinon (JS), gomock (Go), MockK (Kotlin), mockall (Rust), Mox (Elixir)
Yarış durumu	<code>go test -race</code> , loom ve Miri (Rust), ThreadSanitizer (C/C++), jstest (Java), Coyote (C#)
Bulanıklaştırma	<code>go test -fuzz</code> , cargo-fuzz, libFuzzer ve AFL++ (C/C++), Atheris (Python), Jazzer (Java), Jazzer.js (Python), goldie (Go), mneme (Elixir)
Anlık görüntü testi	Jest ve Vitest (dahili), insta (Rust), syrupy (Python), goldie (Go), mneme (Elixir)
Kapsayıcı tabanlı test	Testcontainers ve dil karşılıkları
HTTP testi	Supertest (JS), <code>httptest</code> (Go, dahili), REST Assured (Java), rack-test (Ruby)
Sözleşme testi	Pact, Spring Cloud Contract, Schemathesis
Yük testi	k6, Gatling, Locust, JMeter, Vegeta, wrk
Uçtan uca test	Playwright, Cypress, Selenium, Puppeteer
Statik analiz	staticcheck ve <code>go vet</code> , ESLint ve TypeScript, mypy ve ruff, SpotBugs ve Error Prone, clippy, PHPStan, RuboCop
Biçimsel doğrulama	TLA+, Alloy, Coq, Lean, Isabelle, Dafny, Frama-C, SPARK, Z3

JavaScript ekosisteminin durumu

Node.js ekosisteminde araç eksikliği yoktur; her kategoride olgun seçenekler bulunur. Eksiklikler başka yerlerde dir.

Modül sistemi geçişi

En belirgin sorun modül sistemleri arasındaki geçişin tamamlanmamış olmasıdır. Jest'in CommonJS öncelikli mimarisi, yalnızca ESM olarak yayımlanan paketlerde sorun üretir. Dönüştürme dışlama kalıplarına yapılan eklemeler kalıcı bir çözüm sağlamaz ve her yeni bağımlılıkla yinelenir. Bu sorunun mimariden kaynaklanması, yapılandırma ayarıyla çözülemeyeceği anlamına gelir.

2026 itibarıyla yeni projeler için önerilen çatı Vitest'tir; ESM ve TypeScript desteği doğal, çalışma süresi belirgin biçimde kısadır. Jest mevcut büyük kod tabanları ve belirli ekosistem bağımlılıkları için sürdürülmektedir.

Yerleşik çalıştırıcının olgunluğu

Node.js ile birlikte gelen `node:test` modülü bağımlılık gerektirmez ve başlatma süresi kısadır. Olgunluğu konusunda kaynaklar çelişir: bir kısmı izleme kipi ve anlık görüntü testi gibi özelliklerin eksik olduğunu, çoğu ekip için henüz alternatif oluşturmadığını belirtir; bir kısmı sahte nesne, kapsam ve izleme kipinin kutudan çıktığını belirtir. Sürümler arasında hızlı değişim olduğu için mevcut Node sürümünde doğrulanması gerekir. Üzerinde uzlaşılan nokta, belirgin biçimde daha az özellik sunduğudur.

Bellek tüketimi

Büyük test paketlerinde bellek tüketimi somut bir kısıt üretir. Elli bin testlik bir yükte Jest'in bellek kullanımı standart sürekli tümleştirme makinelerinin sınırlarını zorlayarak daha pahalı makinelere geçişi zorunlu kılabilmektedir; Vitest aynı yükte belirgin biçimde daha az bellek kullanır.

Kararsız testler

Kararsız (*flaky*) testlerin başlıca sebepleri testler arası temizliğin eksikliği, eşzamansız koddaki yarış durumları ve dış servislere yalıtımsız bağımlılıktır. Node tarafında bunun için Go'nun yarış durumu tespit edicisine denk bir yerleşik araç bulunmaz; sorun süreç disipliniyle ele alınır. Maliyeti yüksektir: test çalıştırıcısı değiştirmek sürekli tümleştirmede saniyeler kazandırırken, kararsız uçtan uca test paketlerinin bakımı çeyrek başına mühendis-haftası düzeyinde kayıp üretir.

Bilim Sosyolojisi ve Bilginin Kurumsal Üretimi

Bu alan bilginin ne olduğunu değil, nasıl üretildiğini inceler. Üç sorusu mühendislikte doğrudan karşılık bulur.

Bir teknoloji neden yaygınlaşır

Teknik üstünlük belirleyici etkenlerden yalnızca biridir. Beş etken sayılabilir.

Kurumsal destek. Arkasında sürekli geliştirme, belgelendirme ve topluluk yatırımı yapan bir kuruluş bulunması, tek kişilik projelerin sağlayamayacağı bir süreklilik üretir.

Ağ etkisi. Çok sayıda kişinin bildiği bir teknolojiyi kurumlar tercih eder, çünkü işe alım kolaylaşır; tercih edildikçe daha çok kişi öğrenir. Döngü kendini besler.

Geçiş maliyeti. Yerleşik bir teknolojiden çıkmak pahalıdır. Yeni seçeneğin “daha iyi” olması yetmez; geçiş maliyetini karşılayacak kadar iyi olması gerekir.

Zamanlama. Bir teknolojinin çözdüğü problemin yaygınlaştığı döneme denk gelmesi belirleyicidir.

Anlatılabilirlik. Basit bir cümleyle özetlenebilen fikirler daha hızlı yayılır.

Teknik olarak üstün seçeneklerin kaybettiği örnekler bilinmektedir. Bu olguya *yola bağımlılık* (*path dependence*) denir: geçmişte alınan kararlar, bugün daha iyi bir seçenek bulunsa da onu engeller (David, 1985).

Pratik sonucu, teknoloji seçiminde ekosistemin teknik özelliklerden çoğu zaman daha belirleyici olmasıdır. Ekosistem, belgelendirme, kütüphane sayısı, çözülmüş problem havuzu ve işe alınabilir insan anlamına gelir.

Ölçümün bozulması

Akademide yükselmenin yayın sayısına bağlanması, ölçümün hedefe dönüşmesinin tipik örneğidir. Sonuçları arasında çalışmaların parçalara bölünmesi, olumsuz sonuçların yayımlanmaması ve tekrarlama çalışmalarının yapılmaması sayılır. Psikoloji ve tıp alanlarında yayımlanmış çalışmaların önemli bir bölümünün tekrarlandığında aynı sonucu vermemesi, tekrarlanabilirlik krizi olarak adlandırılmıştır (Open Science Collaboration, 2015).

Karşı önlemler ve mühendislikteki karşılıkları şunlardır:

- **Ön kayıt:** Deney yapılmadan önce yöntem ve analiz planı kayıt altına alınır. Mühendislikteki karşılığı, hata aranmadan önce hipotezin yazılmasıdır.
- **Olumsuz sonucun kaydedilmesi:** Çalışmayan yaklaşımlar da yazılır. Mühendislikteki karşılığı, denenip başarısız olan çözümlerin postmortem’e geçirilmesidir; yazılmadığında aynı deneme tekrarlanır.
- **Tekrarlanabilirlik:** Veri ve kod paylaşılır. Mühendislikteki karşılığı, performans iddialarının ölçüm koduyla birlikte sunulmasıdır; aksi hâlde iddia denetlenemez.

- **Ölçüm çiftleme:** Tek bir gösterge yerine birbirini dengeleyen göstergeler kullanılır.

Yenilik üreten yapılar

Belirli araştırma kurumlarının olağandışı sayıda temel yenilik üretmesi sosyolojik bir inceleme konusudur. Literatürde tekrarlanan ortak özellikler şunlardır:

- Uzun vadeli finansman ve kısa vadeli çıktı baskısının bulunmaması.
- Farklı disiplinlerin fiziksel yakınlığı; kesişim noktalarında ortaya çıkan buluşlar.
- Problem seçiminde araştırmacı özerkliği.
- Üretimle bağlantının bulunması ancak doğrudan tabi olunmaması. Bağlantı tümüyle koptuğunda fikirler ürüne dönüşmez; fazla yakın olduğunda uzun vadeli çalışma yapılamaz.
- Başarısızlığın olağan sayılması.

Bu koşulların bir kısmının olağandışı ekonomik konumlardan kaynaklandığı, dolayısıyla doğrudan taklit edilemeyeceği belirtilmelidir. Küçük ölçekte uygulanabilir kısmı, keşif işinin teslimat işinden ayrılması, keşfin çıktısının “sonuç alınmadı” olabilmesi ve farklı işlerle uğraşan kişilerin aynı ortamda bulunmasıdır.

Diğer Alanlar

Hukuk felsefesi

Bu alanın merkezî problemi, sonsuz sayıda durumun sonlu sayıda kuralla kapsanmasıdır. Aynı problem yetkilendirme sistemleri, iş kuralları ve hesaplama motorları yazılırken karşımıza çıkar.

Kural ve ilke

Kural kesin sınır tanımlar; tartışma bırakmaz ve otomatik uygulanabilir, ancak sınırın hemen iki yanındaki durumlara aynı biçimde davranamaz. **İlke** yoruma açıktır; her duruma uyarlanabilir, ancak öngörülebilirlik ve tutarlılık kaybı üretir.

```
// Kural
if dosyaBoyutu > 10_000_000 { reddet() }

// İlke
if dosyaCokBuyuk(dosya, kullanıcı, baglam) { reddet() }
```

Seçim, durum uzayının öngörülebilirliğine bağlıdır. Durumlar önceden biliniyorsa kural, sürekli yeni durumlar çıkıyorsa ilke tercih edilir. Karma çözüm, kural koyup istisna mekanizması eklemektir.

Açık doku ve boşluk

Hart, kuralların bir *çekirdek* anlamı ile *belirsiz bir kenarı* olduğunu savunur (Hart, 1961). “Parka araç sokulamaz” kuralında otomobil çekirdekte, bisiklet ve kaykay kenardadır.

Hukuk boşluğu kabul eder ve yorum mekanizması kurar. Yazılımda boşluk kabul edilmez ve karşılanmayan durum hata üretir. Uygulanabilir kural, kural yazılırken kapsanmayan durumun davranışının baştan belirlenmesidir: reddetme, izin verme ya da insan kararına yönlendirme.

Geriye yürümezlik

Hukukta genel ilke, yeni kuralın geçmiş işlemlere uygulanmamasıdır. Yazılımdaki karşılığı veri göçü kararlarıdır: iş kuralı değiştiğinde eski kayıtların yeniden hesaplanması, eski kuralla bırakılması ya da tarih bazlı ayırım yapılması seçenekleri vardır. Vergi yazılımlarının yıl bazlı kural kümeleri tutması ikinci seçeneğin uygulanmış hâlidir.

Yorum yöntemleri

Hukukta dört yorum yöntemi ayırt edilir: lafzi (kelimelerin anlamı), amaçsal (kuralı koyanın amacı), sistematik (diğer kurallarla birlikte anlamı) ve tarihsel (konduğu dönemdeki anlamı).

Eski kod okurken aynı dört soru sorulur: kod ne yapıyor, yazan neyi çözmeye çalışıyordu, sistemin geri kalanıyla nasıl uyuyor, o dönemde hangi kısıtlar vardı. Sebebi bilinmeyen bir yapıyı kaldırmadan önce neden konduğunun araştırılması gerektiğini söyleyen ilke, tarihsel yorumun mühendislik karşılığıdır.

Siyaset felsefesi

Meşruiyet

Bir kararın bağlayıcı sayılmasının gerekçesi üç biçimde verilir: rıza, sonuçların iyiliği ve usule uygunluk.

Dağıtık sistemlerde aynı soru sorulur: bir düğümün kendini lider ilan etmesi neden diğerleri için bağlayıcıdır? Uzlaşma protokollerinin cevabı usule dayanır: seçim yapılmış, çoğunluk sağlanmış, dönem numarası geçerlidir. Aynı sorunlar da tekrarlanır; iki düğümün aynı anda liderlik iddia etmesi durumu, siyasetteki iç bölünmenin yapısal karşılığıdır.

Toplu karar ve sınırları

Condorcet paradoksu, üç kişilik bir grupta bireysel tercihler geçişli olsa bile toplu tercihin döngüsel olabileceğini gösterir. Arrow teoremi bunu genelleştirir: makul kabul edilen bir dizi koşulu aynı anda sağlayan bir toplu tercih yöntemi yoktur (Arrow, 1951).

Pratik sonucu, üç ve daha fazla seçenek arasında yapılan oylamalarda oylama sırasının sonucu etkileyebilmesidir. Bu bir usulsüzlük değil, matematiksel bir zorunluluktur.

Merkezî ve dağıtık yapı

Merkezî yapı hızlı ve tutarlı karar üretir, ancak tek nokta arızasına açıktır. Dağıtık yapı yavaş karar üretir ve tutarsızlık riski taşır, ancak dayanıklıdır. Tek parçalı ve servis tabanlı mimari tartışması bu yapının aynısıdır ve cevabı bağlama bağlıdır.

Yetki devri

Kimin neye karar verebileceği sorusu, yetkilendirme sistemlerinde doğrudan kodlanır. Rol tabanlı ve öznitelik tabanlı erişim denetimi ile yetkilendirme kapsamı birer yetki devri mekanizmasıdır. Aynı sorular sorulur: yetki devredilebilir mi, devredilen yetki geri alınabilir mi, alt yetki üst yetkiyi aşabilir mi.

Özgürlük kavramının iki anlamı

Negatif özgürlük engelin bulunmamasıdır; **pozitif özgürlük** yapabilme imkânının bulunmasıdır (Berlin, 1958).

Arayüz tasarımıındaki karşılığı şudur: her işleme izin veren ancak anlaşılamayacak kadar karmaşık bir arayüz negatif anlamda özgür, pozitif anlamda kısıtlıdır.

Sınırlı ancak kullanılabilir bir arayüz pratikte daha fazla işin yapılmasını sağlayabilir. Esnek ve görüş dayatan çatı kütüphaneleri arasındaki tercih bu ayrımın uygulamasıdır.

Toplum felsefesi

Conway yasası

Bir sistem tasarlayan kuruluş, kaçınılmaz olarak kendi iletişim yapısını yansıtan bir tasarım üretir (Conway, 1968). Sebebi sosyolojiktir: kolay iletişim kuran birimlerin ürettiği bileşenler sıkı bağlı olur, zor iletişim kuranlar araya arayüz koyar.

Pratik uygulaması ters yönde çalıştırmaktır: istenen mimari yapıya uygun bir organizasyon önce kurulur. Birbirine bağımlı ekiplerin ürettiği servisler, ayrı yayımlanabilir olsalar bile birlikte yayımlanmak zorunda kalır.

Koordinasyon problemleri

Bireysel olarak makul davranışlar toplu olarak kötü sonuç üretebilir.

Ortak alan trajedisi: Ortak kaynak herkes tarafından kullanılır, kimse tarafından bakılmaz. Ortak kütüphaneler ve paylaşılan veritabanı şemaları bu kalıba girer.

Bedavacılık: Test yazma, belge güncelleme ve teknik borç ödeme gibi işler herkesin yararına olduğu hâlde kimsenin önceliği değildir. Çözüm teknik değil örgütseldir: sahiplik atanır.

Karşılıklı bekleme: Birbirine bağımlı iki birim kendi işini önceliklendirdiğinde ikisi de ilerleyemez.

Kurumlar ve örtük kurallar

Kurum, tekrarlanan davranış kalıbıdır ve yazılı olmak zorunda değildir. Ekip içi kurallar bu anlamda birer kurumdur ve yeni katılan kişi bunları belgede bulamaz, davranışı gözleyerek öğrenir. Bu, örtük bilginin kurumsal biçimidir ve yazılı kuralın değiştirilmesinin davranışı değiştirmemesinin sebebidir.

Kavramlar tarihi

Bugün doğal görünen kavramların çoğu belirli bir dönemin ürünüdür ve kavram değiştiğinde düşünme biçimi de değişir.

“Verimlilik” sanayi dönemine ait bir kavramdır; öncesinde “iyi iş” vardı, ölçülebilir verim yoktu. “Kullanıcı” kavramı insanı sistemle etkileşen bir birime indirger; bu kavram kullanıldığında bazı sorular sorulur, bazıları sorulmaz. Yazılım hatalarının “böcek” olarak adlandırılması, hatayı dışarıdan gelen bir olgu gibi sunar. “Mimari” terimi inşaatın alınıp alınmadığı ve temel, katman, plan gibi çağrışımlar taşır; ancak yazılımın sürekli değiştiği, binanın değişmediği farkını gizler.

Buradan çıkan uygulanabilir alışkanlık, kendi alanının merkezi kavramlarının sorgulanmasıdır: “teknik borç” metaforu borcun ödenip bittiği çağrışımını taşır, oysa karşılığı böyle davranmaz.

Aydınlanma düşüncesi

On yedinci ve on sekizinci yüzyıllarda yerleşen bir dizi varsayım bugün varsayım olarak görünmemektedir: aklın her problemi çözebileceği, doğanın anlaşılabilir ve denetlenebilir olduğu, ilerlemenin sürekli olduğu, geleneğin yerine kanıtın geçmesi gerektiği ve otorite yerine bireysel akla güvenilmesi.

Bu varsayımların her biri sonradan tartışmaya açılmıştır. Her problemin çözülebilirliği hesaplanabilirlik teoremleriyle sınırlanmıştır. İlerlemenin doğrusallığı Kuhn tarafından sorgulanmıştır.

Yazılım kültüründeki yankıları doğrudandır: her sürecin otomatikleştirilebileceği varsayımı, ölçülemeyenin yönetilemeyeceği sloganı, yeni olanın iyi olduğu kabulü ve yeterince düşünüldüğünde her problemin çözüleceği inancı.

Frankfurt Okulu’nun eleştirisi bu bağlamda ayrıca anılmalıdır: aklın araçsallaşması, “nasıl” sorusunun sorulup “niçin” sorusunun sorulmaması sonucunu doğurur (Horkheimer ve Adorno, 1944).

Ders Kümelerinin Değerlendirilmesi

Öncelik sıralaması

Mühendislik pratiğine sağladığı doğrudan karşılığa göre sıralama:

Ders kümesi	Sağladığı
Modern / Sembolik Mantık I-II	Doğruluk tablosu, De Morgan, niceleyici ayrımı, meta-teorik sınır
Eleştirel Düşünme	Argüman çözümlemesi, kanıt gücü, çürütme yöntemi
Bilim Felsefesi	Yanlışlanabilirlik, Duhem-Quine, Lakatos ölçütü, nedensellik
Bilgi Felsefesi	Gettier durumu, sorgulanmayan taban, bilgi kaybı
Dil Felsefesi	İsimlendirme, kavram çakışması, boş gösterici
Hukuk Felsefesi	Kural yazımı, boşluk yönetimi, geriye yürümezlik
Bilim Sosyolojisi	Kurumsal bilgi üretimi, ölçüm bozulması
Siyaset Felsefesi	Meşruiyet, toplu karar sınırları, yetki devri
Toplum Felsefesi	Conway yasası, koordinasyon problemleri
Klasik Mantık	Sınıflandırma bütünlüğü, tarihsel arka plan
Metafizik / Varlık Felsefesi	Kimlik-değer ayrımı, zaman modeli, parça-bütün

Tekrar eden dersler

Bir müfredatta aynı içeriğin farklı kodlarla bulunması olağandır. “Modern Mantık” ile “Sembolik Mantık” eşanlamlıdır; “Bilgi Felsefesi” ile “Bilgi Teorisi” eşanlamlıdır. Aynı alanın lisans, yüksek lisans ve doktora sürümleri arasındaki fark içerik değil derinlik ve çıktı düzeyindedir: lisansta kavramlar öğrenilir, yüksek lisansta çözülmemiş tartışmalar okunur, doktora tartışmaya katkı beklenir.

Bir istisna, doktora düzeyindeki bilim felsefesi dersinin nedensellik teorilerini ayrı bir başlık olarak içermesidir; bu konunun mühendislik karşılığı diğer düzeylerde bulunmaz.

Mühendislik karşılığı zayıf olan dersler

Esikçağ felsefesi tarihi, İslam felsefesi tarihi ve ekolleri, Osmanlı dönemi düşünce dersleri, Alman idealizmi, din felsefesi, kültür felsefesi, duygu tarihi, film felsefesi ve dönemsel düşünce tarihi dersleri bu kümededir. Bu derslerin entelektüel değeri ayrı bir konudur; burada söylenen yalnızca mesleki gerekçeyle seçilmelerinin savunulamayacağıdır.

Aristoteles'in mantık ve sınıflandırma katkıları bu değ erlendirmenin dıřındadır ve ilgili b l mlerde ayrıca ele alınmıřtır.

Okuma Listesi

Öncelikli

- Kuhn, *Bilimsel Devrimlerin Yapısı*. Paradigma kavramı ve devrim modeli. Kısa ve okunabilir.
- Polanyi, *Örtük Boyut*. Nasıl bilgisi ve belgelendirmenin sınırı.
- Dekker, *İnsan Hatasını Anlamak*. Kök neden arayışının sınırları ve suçlamasız inceleme.
- Popper, *Conjectures and Refutations*. Yanlışlanabilirlik ölçütünün okunabilir sunumu; *Bilimsel Araştırmanın Mantığı* daha ağırdır.
- Hofstadter, *Gödel, Escher, Bach*. Eksiklik teoreminin erişilebilir açıklaması.

İkincil

- Lakatos, araştırma programları üzerine makaleleri. İlk uzun makale yeterlidir.
- Hume, *İnsanın Anlama Yetisi Üzerine Bir Soruşturma*. Tümevarım problemi.
- Quine, “İki Dogma” makalesi. Duhem–Quine tezinin kaynağı.
- Perrow, *Normal Kazalar*. Karmaşık sistemlerde arızanın yapısal kaçınılmazlığı.
- Brooks, *The Mythical Man-Month*. Yazılım karmaşıklığı üzerine klasik.

Antik kaynaklar

Antik Yunan kaynaklarının katkısı, cevapları vermek değil soruları kurmaktır. Doğrudan karşılığı olan üç metin:

- Aristoteles, *İkinci Analitikler*. Bilmenin sebebini bilmek olduğu tezi ve ispat zincirinin nerede duracağı problemi.
- Aristoteles, *Kategoriler*. Tanım ve sınıflandırma kuralları.
- Öklid, *Elementler*, birinci kitap. Aksiyomdan sistem kurma yöntemi; şartname yazımının modeli.

Kavramsal değeri olan üç metin daha eklenebilir: Platon’un *Menon* diyalogu (bilginin değeri), *Theaitetos* diyalogu (bilgi tanımının üç denemesi ve başarısızlıkları) ve Sextus Empiricus’un şüphecilik derlemesi (gerekçe zinciri üçlemesi). Arşimet’in yöntem üzerine metni, keşif ile ispatın ayrı süreçler olduğunu açıkça belirtmesi bakımından dikkate değerdir.

Antik kaynakların sınırı yöntemdedir: deney ve ölçüm bulunmadığı için pek çok olgusal iddia yüzyıllarca sınanmadan kalmıştır. Bu notta ele alınan konuların büyük kısmı on yedinci yüzyıl sonrasına aittir.

Sonuç

Felsefe müfredatının mühendislik açısından değerlendirilmesinde iki farklı katkı türü ayrılmalıdır. Birinci tür, mühendisin zaten sezgisel olarak uyguladığı ayrımları adlandırır. Bu katkı küçümsenmemelidir, çünkü adı olan hata daha az tekrarlanır; ancak yeni bilgi vermez. İkinci tür, sezgiyle ulaşılamayacak sonuçlar sağlar: hesaplanabilirlik sınırları, doğrulama ilişkisinin biçimsel paradoksları, ilerleyen ve çürüten program ayrımı bu türdendir.

Bu notta kurulan ayrımların pratik karşılığı birkaç uygulanabilir kurala indirgenebilir. Bir iddia kabul edilmeden önce onu çürütecek gözlem sorulur. Bir hata kapatılmadan önce sebebi yazılabilir olmalıdır. Bir hipotez kurulduğunda aynı veriye uyan ikinci hipotez aranır ve ikisini ayıran gözlem belirlenir. Bir ölçüm hedefe dönüştürülmez, dengeleyicisiyle birlikte izlenir. Bir mimari iş önerildiğinde daha önce mümkün olmayan neyi mümkün kılacağı sorulur.

Çerçevenin sınırları da belirtilmelidir. Uygulanabilirlik ölçütü, bu notun kendi seçtiği ölçüttür ve felsefenin değerini tüketmez; burada düşük öncelikli sayılan derslerin başka ölçütlere göre değeri yüksek olabilir. İkinci sınır, kurulan karşılıkların bir kısmının yapısal benzerlikten ibaret olmasıdır: dağıtık uzlaşma protokolleri ile siyasal meşruiyet arasındaki benzerlik gerçektir, ancak biri diğerinin uygulaması değildir. Üçüncü sınır, araç ekosistemine ilişkin bilginin hızla eskimesidir; kategori haritası kalıcıdır, araç adları değildir.

Açık kalan iki soru vardır. Birincisi, bilgi boşluğunun ölçülebilirliği: sistemin çalıştığı ancak neden çalıştığının bilinmediği durumun kriz anından önce tespit edilmesini sağlayan bir gösterge kümesi kurulabilir mi? İkincisi, Lakatos ölçütünün kısa vadede uygulanabilirliği: bir mimari işin yeni bir imkân açıp açmadığı, çoğu zaman ancak aylar sonra anlaşılır; karar anında kullanılabilecek erken göstergeler nelerdir?

Son olarak, buradaki bütün başlıkların ortak bir yapısı bulunduğu belirtilmelidir. Mantık geçerli çıkarımın sınırını çizer, bilim felsefesi kanıtın gücünü sorgular, bilgi felsefesi gerekçenin yeterliliğini sorar, dil felsefesi iddianın neye gönderme yaptığını inceler. Dördü de aynı soruyu farklı düzeylerde sorar: bir iddiaya güvenmek için elimizde ne var?

Kaynaklar

- Arrow, K. J. (1951). *Social Choice and Individual Values*. Wiley.
- Austin, J. L. (1962). *How to Do Things with Words*. Oxford University Press.
- Berlin, I. (1958). *Two Concepts of Liberty*. Oxford University Press.
- Claessen, K. ve Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *ICFP*.
- Conway, M. E. (1968). How Do Committees Invent? *Datamation*.

- David, P. A. (1985). Clio and the Economics of QWERTY. *American Economic Review*.
- Dekker, S. (2002). *The Field Guide to Human Error Investigations*. Ashgate.
- Dijkstra, E. W. (1970). Notes on Structured Programming.
- Duhem, P. (1906). *La Théorie Physique: Son Objet, Sa Structure*.
- Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley.
- Frege, G. (1879). *Begriffsschrift*.
- Frege, G. (1892). Über Sinn und Bedeutung.
- Gettier, E. (1963). Is Justified True Belief Knowledge? *Analysis*.
- Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I.
- Goodhart, C. (1975). Monetary Relationships: A View from Threadneedle Street. Yaygın formülasyonu Strathern (1997) tarafından verilmiştir.
- Goodman, N. (1955). *Fact, Fiction, and Forecast*. Harvard University Press.
- Grice, H. P. (1975). Logic and Conversation. *Syntax and Semantics*.
- Hanson, N. R. (1958). *Patterns of Discovery*. Cambridge University Press.
- Hart, H. L. A. (1961). *The Concept of Law*. Oxford University Press.
- Hempel, C. G. (1945). Studies in the Logic of Confirmation. *Mind*.
- Hempel, C. G. ve Oppenheim, P. (1948). Studies in the Logic of Explanation. *Philosophy of Science*.
- Hoare, C. A. R. (2009). Null References: The Billion Dollar Mistake. QCon konuşması.
- Horkheimer, M. ve Adorno, T. W. (1944). *Dialektik der Aufklärung*.
- Hume, D. (1748). *An Enquiry Concerning Human Understanding*.
- Kuhn, T. S. (1962). *The Structure of Scientific Revolutions*. University of Chicago Press.
- Lakatos, I. (1970). Falsification and the Methodology of Scientific Research Programmes.
- Laudan, L. (1981). A Confutation of Convergent Realism. *Philosophy of Science*.

- Lewis, D. (1973). Causation. *Journal of Philosophy*.
- Newcombe, C. ve diğeri (2015). How Amazon Web Services Uses Formal Methods. *Communications of the ACM*.
- Open Science Collaboration (2015). Estimating the Reproducibility of Psychological Science. *Science*.
- Polanyi, M. (1966). *The Tacit Dimension*. Routledge.
- Popper, K. (1934). *Logik der Forschung*.
- Quine, W. V. O. (1951). Two Dogmas of Empiricism. *The Philosophical Review*.
- Rice, H. G. (1953). Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the AMS*.
- Russell, B. (1905). On Denoting. *Mind*.
- Ryle, G. (1949). *The Concept of Mind*. Hutchinson.
- Stevens, S. S. (1946). On the Theory of Scales of Measurement. *Science*.
- Thompson, K. (1984). Reflections on Trusting Trust. *Communications of the ACM*.
- Toulmin, S. (1958). *The Uses of Argument*. Cambridge University Press.
- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the LMS*.
- Wittgenstein, L. (1953). *Philosophische Untersuchungen*.

Test araçlarına ilişkin bölümdeki güncel durum bilgileri 2026 yılına ait çevrimiçi karşılaştırma ve araç belgelerine dayanmaktadır; bu tür bilgiler hızla eskidiği için kritik kararlarda yeniden doğrulanmalıdır. Bell Labs ve benzeri araştırma kurumlarının örgütsel özelliklerine ilişkin genellemelerin literatürdeki kaynakları ayrıca doğrulanmalıdır.